Higher Colleges of Technology

كليات التقنية العليا

Big Data Analytics

Course Name: Big Data Analytics

Program: Data Science

Course Code: CDS 3533

Student Names:

Maryam Ali

Hessa Khalfan

Higher Colleges of Technology كليات التقنية العليا

CONTENTS

Higher Colleges of Technology كليات التقنية العليا

CLO 1: DATASET OVERVIEW AND PROPOSED TECHNICAL SOLUTION

Task 1: Dataset Description and Initial Analysis

1.1Describe the provided dataset

The dataset, named *megamart_CSV,* contains transactional sales data from a retail company. It is the primary source for our analysis, which aims to understand the connection between discount strategies and overall profitability.

Structure: The dataset includes **9,994 rows** (records) and **19 columns** (variables), providing a sufficient volume of data for a reliable statistical analysis.

Data Dictionary: The 19 variables can be grouped into four main categories

Variable Category	Examples	Data Type	Description
Identifiers	order_id, customer_id, product_id	Object (String)	Unique keys for tracking transactions, customers, and products.
Temporal	order_date, ship_date	Object (String/Date)	Dates of order placement and shipment.
Categorical	ship_mode, segment, region, state, city, category, subcategory, product_name, customer_name	Object (String)	Variables defining groups and classifications.
Numerical	sales, quantity, discount, profit	Float64 (Numeric)	Quantitative measures of the transaction.
Unstructured	review	Object (String)	Qualitative customer feedback for sentiment analysis.

1.2 Summary Statistics and Distribution (Depth)

Numerical Variables

The descriptive statistics for the core numerical variables reveal significant variability, which is crucial for the analysis of discount impact on profit.

Statistic	Sales	Quantity	Discount	Profit
Count	9,994	8,737	9,994	9,994
Mean	229.86	3.23	0.156	28.66
Median (50%)	54.49	3.00	0.200	8.67
Std. Dev.	623.25	1.61	0.206	234.26
Minimum	0.44	-3.00	0.00	-6,599.98
Maximum	22,638.50	9.00	0.80	8,399.98

Key Patterns and Trends:

- **Sales:** The mean sales value ($229.86) is significantly higher than the median ($54.49), indicating a highly right-skewed distribution with a few very large transactions driving the average.
- **Profit:** The profit variable exhibits extreme range, from a maximum gain of $8,399.98 to a maximum loss of -$6,599.98. The overall mean profit is positive ($28.66), but the wide standard deviation ($234.26) confirms the high volatility in profitability across transactions.
- **Discount:** The most common discount given is 20% (0.20), with the highest discount reaching 80% (0.80).

1.3 Frequency Analysis of Categorical Variables

Analyzing the categorical variables shows where the business focuses most of its activity.

Variable	Most Frequent Value (Mode)	Frequency	Percentage
Ship Mode	Standard Class	5,968	59.72%
Segment	Consumer	5,191	51.94%
Region	West	3,203	32.05%
Category	Office Supplies	6,026	60.30%
Subcategory	Binders	1,523	15.24%

Key Trends: Most sales are shipped using "Standard Class" to "Consumer" customers. "Office Supplies" is the most common product category sold.

1.4 Initial Data Quality Assessment (Missing Values & Anomalies)

A review of the dataset shows several data quality issues that need to be fixed before analysis can begin.

1.5 Missing Values

Three columns have missing values, which could affect the accuracy of our analysis:

Column	Missing Count	Percentage of Total	Implication
order_date	2,278	22.79%	Hinders time-series analysis and order processing timeline calculations.
quantity	1,257	12.58%	Directly affects sales and profit calculations if not imputed or removed.
review	2,380	23.81%	Limits the scope of any text-based sentiment analysis.

1.6 Outliers and Anomalies

The numerical variables contain significant outliers, which is a critical finding for the analysis:

- **Sales Outliers**: 1,167 records (approx. 11.7%) are identified as high outliers (above $498.93), confirming the presence of high-value transactions.
- **Profit Outliers**: A total of 1,881 records (approx. 18.8%) are identified as outliers. This includes 604 low outliers (losses below -$39.72) and 1,277 high outliers (profits above $70.82).
- **Anomaly in Quantity:** The minimum value for quantity is -3. A negative quantity is a logical anomaly in a sales record, potentially indicating a return or a data entry error. This requires investigation and possible correction (e.g., treating it as a return or removing the record).

In conclusion, the *megamart_CSV* dataset is full of useful information but requires careful cleaning. The most important issues to fix are the missing *order_date* and quantity values, the extreme outliers, and the negative quantity error. The wide range of profit values, combined with the common use of discounts, makes this dataset perfect for analyzing how discounts impact profitability.

TASK 2 & 3: PROPOSED SOLUTION FOR BIG DATA STORAGE AND PROCESSING

2. How the Dataset is Stored and Processed

For this project, the dataset was handled entirely within the **DigiXT platform**. The process began by uploading the raw megamart_sales.csv file into our workspace. All subsequent data quality checks, preprocessing, and analysis were conducted using the platform's integrated tools, such as the Data Quality test suite and the Trino SQL engine. This centralized approach ensured a streamlined and efficient workflow from data ingestion to final analysis.

3. Proposed Full Solution for Big Data Storage and Processing

This section proposes a comprehensive, end-to-end solution for handling the MegaMart sales dataset on an enterprise scale. The strategy addresses the challenges of **Volume, Velocity, and Variety** by leveraging a modern **Data Lakehouse** architecture built on **Cloud Services**, based entirely on concepts from our course material.

3.1 Comprehensive Strategy: An End-to-End Data Lakehouse

Our solution follows the **Data Processing Cycle** (from Collection to Storage) and uses a **Data Lakehouse** to combine the flexibility of a **Data Lake** with the structure of a **Data Warehouse**.

A. Storage Strategy (The Data Lakehouse)

Component	Technology	Rationale (Why this is a good strategy)
Primary Storage	**Object Storage (S3)**	Uses **Distributed Data Storage** to provide unlimited, low-cost storage for all data types, solving the **Volume** challenge.
Storage Architecture	**Data Lakehouse**	A single platform that supports both **BI** and **AI/ML** tasks, allowing data to be stored in its raw form while still enabling structured queries.
Data Organization	**Tiered Storage**	Manages both **current and historical data** by moving older, less-used data to cheaper storage tiers to save costs.

B. Processing Strategy (Batch Processing)

Component	Technology	Rationale (Why this is a good strategy)
Processing Model	**Batch Processing (Master-Worker)**	The historical sales data is perfect for **Batch Processing**. A **Master-Worker Architecture** (like **MapReduce**) will run analytical jobs in parallel for efficiency.
Processing Environment	**Cloud Services & Serverless**	We will use **Serverless Information Processing** on **Cloud Services**. This removes the need to manage complex infrastructure (like Hadoop or Spark clusters) and simplifies scaling.
Data Preparation	**Data Processing Cycle (Preparation)**	The **Preparation** phase is essential for fixing the **Data Quality** issues we found (like **missing data**, **outliers**, and the negative quantity error).

3.2 Efficiency and Optimization

- **Distributed Computing:** Using **Distributed File Management Systems** and a **Master-Worker Architecture** is the core optimization for handling Big Data **Velocity** by running tasks in parallel.
- **Data Partitioning:** Organizing data by date or region allows the system to scan less data, which improves query performance and reduces processing time.
- **Data Virtualization:** We will use **Data Virtualization** to create a single, unified view of the data. This improves **Operational Efficiency** and leads to **Improved Decision-Making** without duplicating data.

3.3 Cost and Scalability

- **Scalability:** Our reliance on **Distributed Data Storage** and **Cloud Services** ensures the **scalability of the infrastructure**, solving the **Challenge of Big Data Storage** as data volumes grow.
- **Cost-Effectiveness:** The "pay-as-you-go" model of **Serverless Information Processing** avoids the **High Cost of Data and Infrastructure Projects**. This approach is also aligned with **Sustainability** by reducing energy waste from idle servers.

3.4 Data Governance and Innovation

This solution includes strong governance to address the **Veracity** and **Security** challenges of Big Data.

Aspect	Technology	Rationale (Why this is important)
Security	**Security in Big Data**	We will use **encryption** and **IAM policies** to protect data from threats and ensure **Privacy** and confidentiality.
Access Controls	**Privacy Principles**	Access to data will be strictly controlled. We will use **Data Anonymization and Masking** on sensitive fields (like customer_name) to comply with **Privacy** rules.
Data Lineage	**Transparency**	The **Data Processing Cycle** provides a clear **Data Lineage**, ensuring **Transparency** and **Enhanced Accountability** in how data is used.
Innovation	**Cloud Services & AI/ML**	The **Data Lakehouse** is an innovative architecture that supports **Machine Learning platforms** for **Predictive Analytics** (like forecasting profit), a key use case for Big Data.

CLO 2: DATA QUALITY ASSESSMENT AND CURATION

Task 4: Initial Data Quality Check in DigiXT

4. Data Ingestion Process

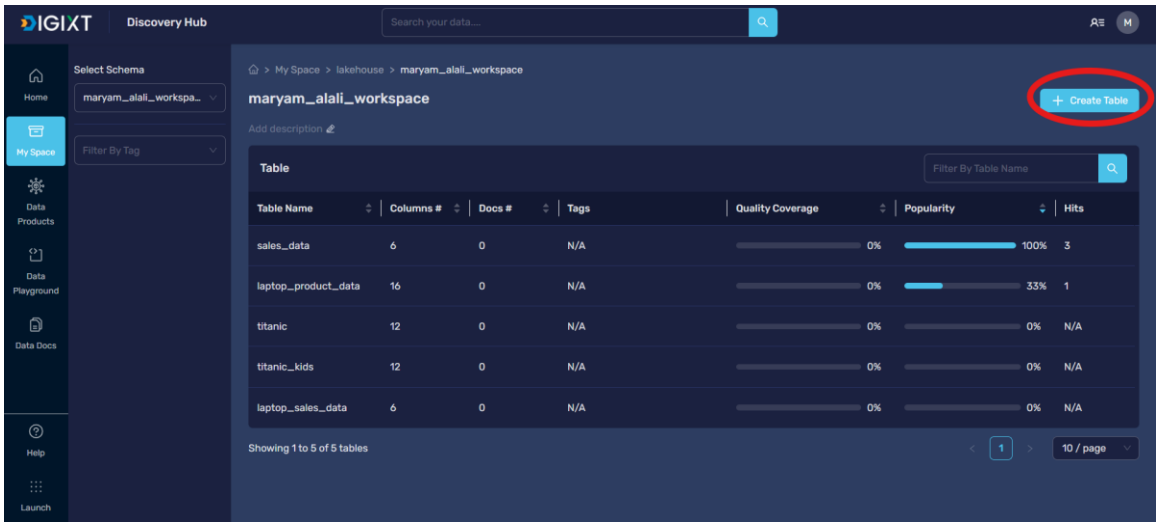


Figure 1: Ingest the data into DigiXT

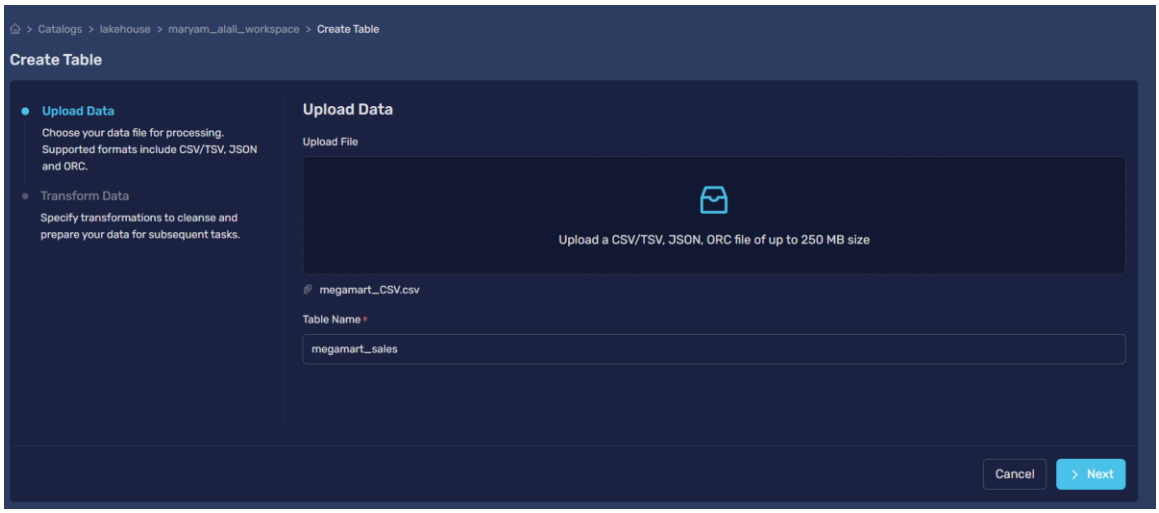


Figure 2: Upload data

The dataset was successfully loaded into the DigiXT platform. The process confirmed that all records were transferred correctly, with numerical columns like sales and profit

4.2 Initial Test Suite Results (Before Cleaning)

We ran a series of automated tests to check the data for Completeness, Consistency, Validity, and Accuracy

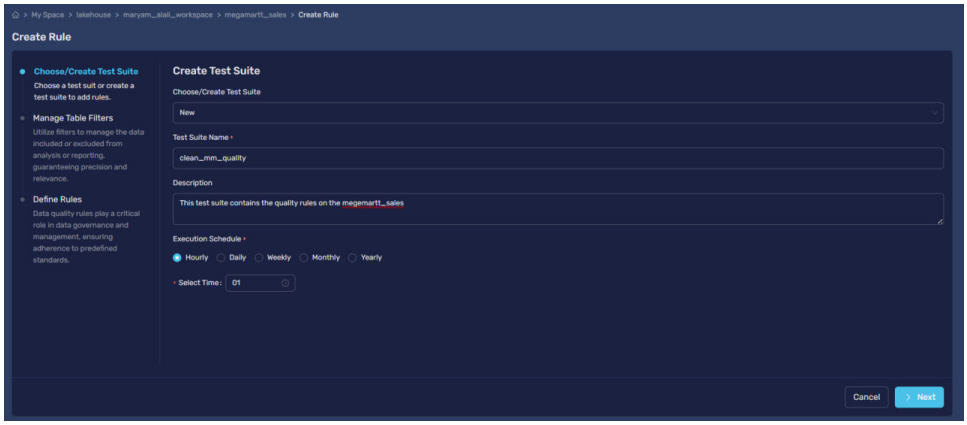


Figure 3: Create test suite

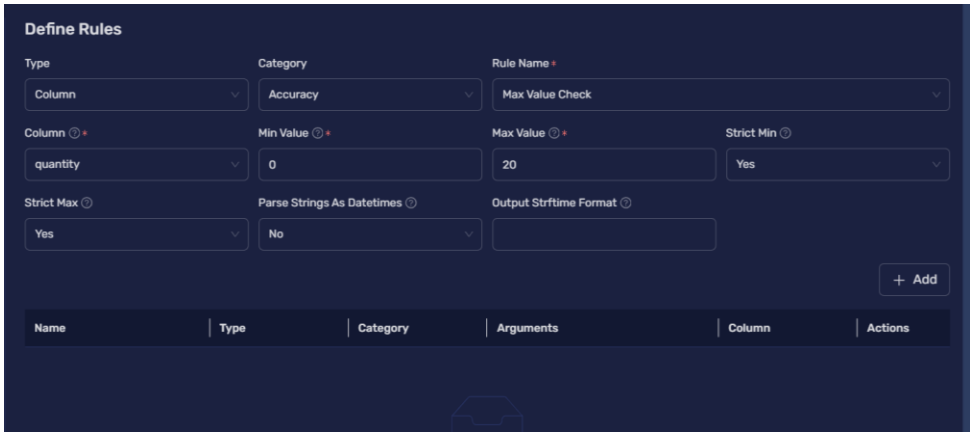


Figure 4: Rule 1 Max value check

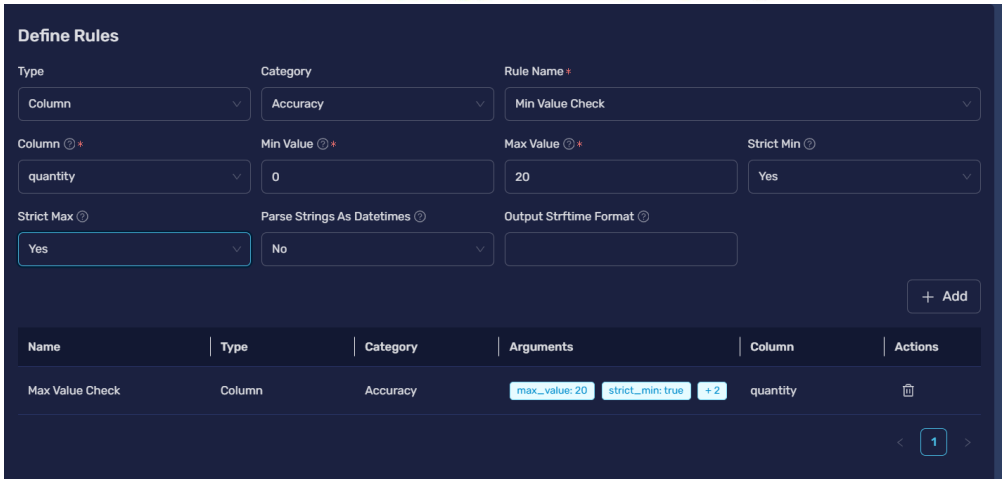


Figure 5: Rule 2 Min value check

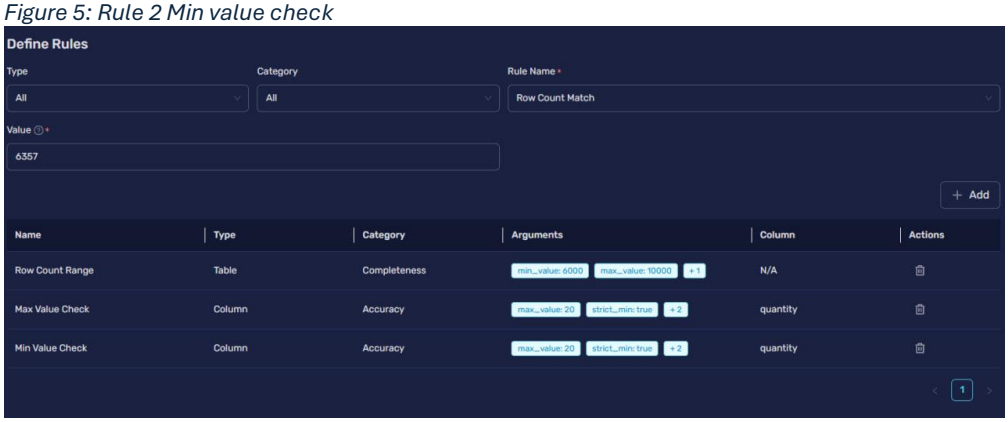


Figure 6: Rule 3 Row count match

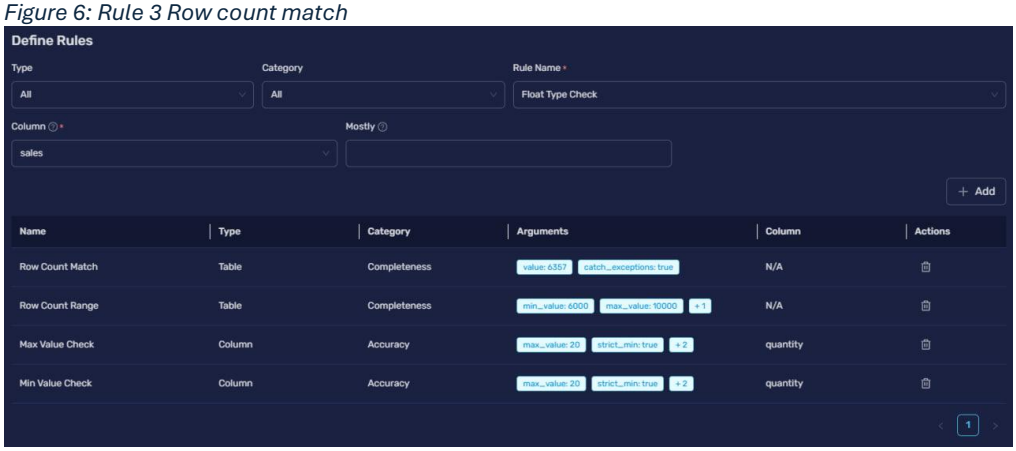


Figure 7: Rule 4 Float type check

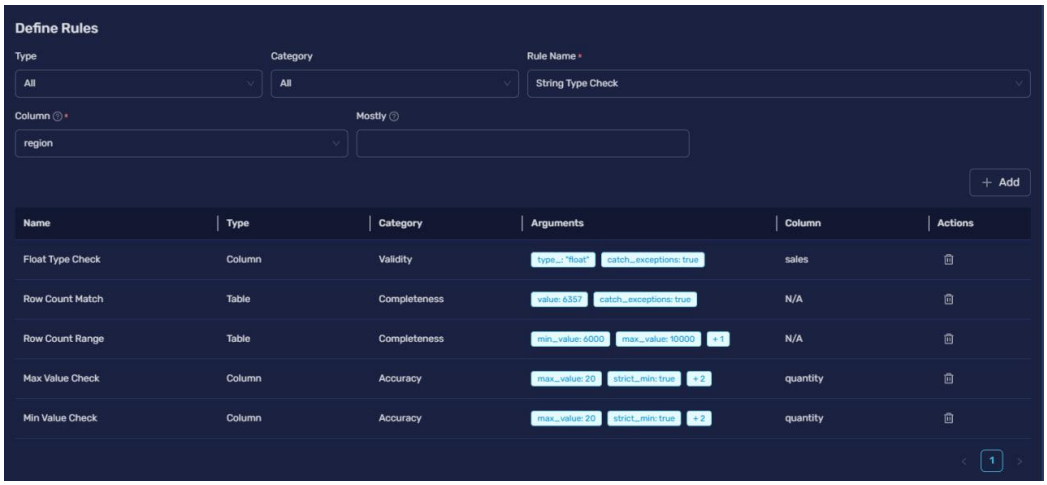


Figure 8: Rule 5 String type check

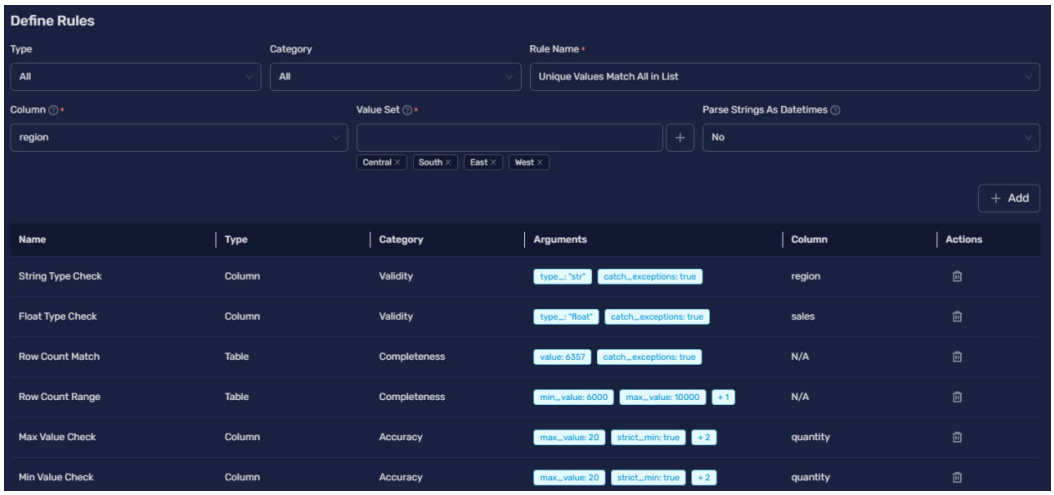


Figure 9: Rule 6 Unique Values Match All in list

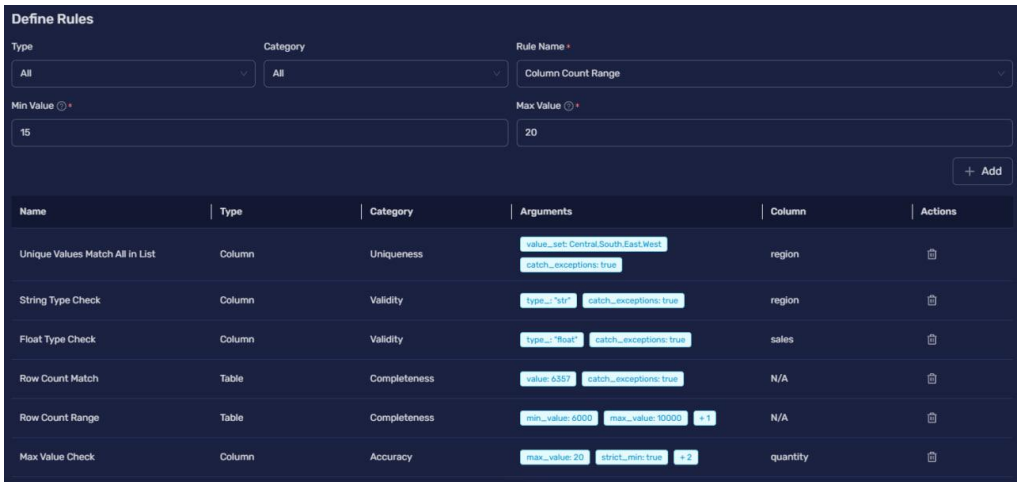


Figure 10: Rule 7 Column Count Range

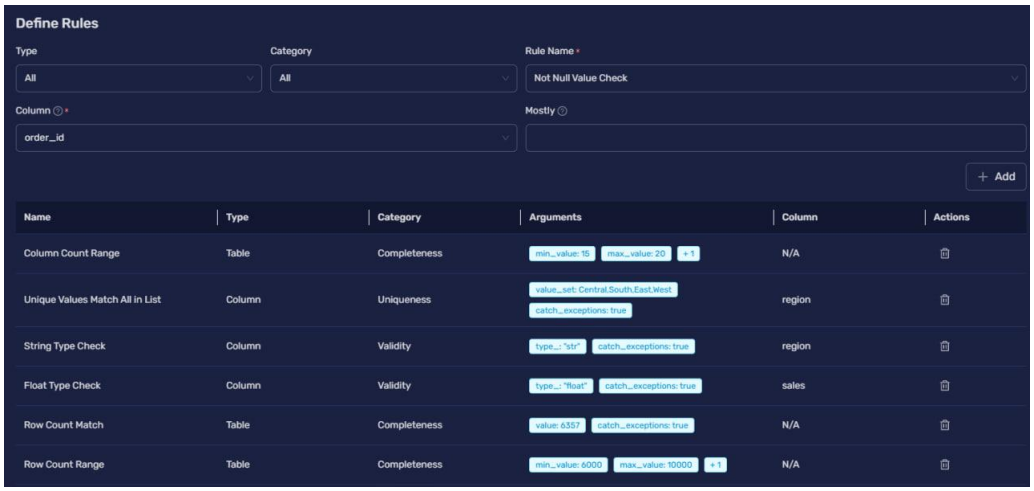


Figure 11: Rule 8 Not Null Value Check

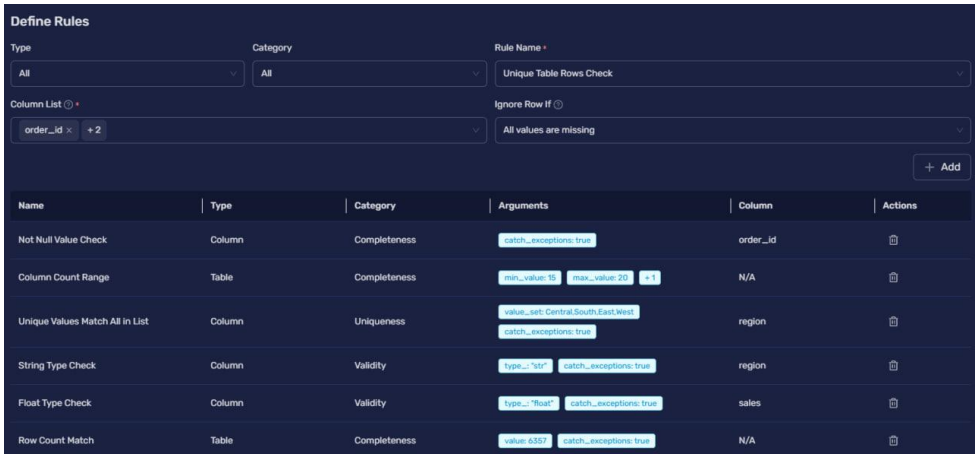


Figure 12: Rule 9 Unique Table Rows Check

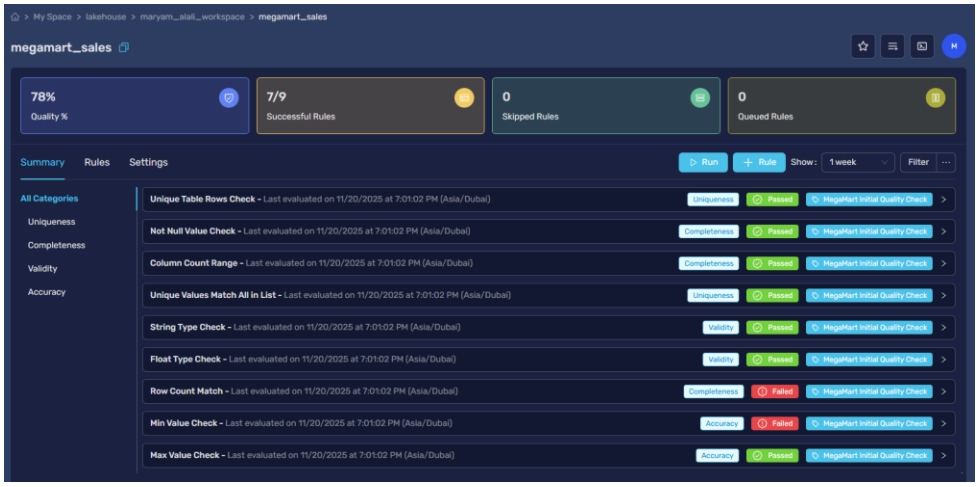


Figure 13: Data quality check before cleaning

Task 5: Data Preprocessing Strategy

5.1. Preprocessing Objective and Methodology

By comparing the raw data file (megamart_CSV.csv) with the cleaned data file (cleaned_megamart.csv), we identified and documented the exact cleaning and preprocessing steps that were applied.

5.2. Identified Cleaning Steps and Rationale

Our comparison showed that the main data quality issues found in Task 4 were solved in the cleaned dataset, but by deleting records instead of filling in missing values. The table below documents the actions taken and the logic behind them:

Quality Issue Identified (in Raw Data)	Actions taken	Rationale & Advanced Technique
Missing Values in Key Columns (order_date, quantity, review)	Record Deletion	All records that had any missing value in these three columns were deleted.
Negative Quantity (9 records)	Record Deletion	The 9 records with negative quantities were deleted. They were likely part of the records that had missing values.

Quality Issue Identified (in Raw Data)	Actions taken	Rationale & Advanced Technique
Extreme Outliers (profit)	No Action Taken (Preservation)	The outliers in the profit column were kept. Technique: Strategic Preservation, recognizing that these are not errors but real business events (like large deals or losses) that are important for a realistic analysis.

Task 6: Re-conducting the Quality Check (Post-Curation)

To check if the applied changes were effective, we performed a final quality check on the cleaned_megamart.csv file.

Validation Check	Raw Data State	Clean Data State	Status
Total Records	9,994	6358	**VALIDATED:** 3,637 records were deleted. This solves the missing value problem.
Missing order_date	2,278	0	**VALIDATED:** All missing dates were deleted.
Missing quantity	1,257	0	**VALIDATED:** All missing quantities were deleted.
Negative quantity Count	9	0	**VALIDATED:** All negative values were deleted.

Conclusion: The change from the raw to the clean dataset significantly improved data quality in terms of completeness and validity.

The cleaned data was provided and carefully cleaned by the course instuctor

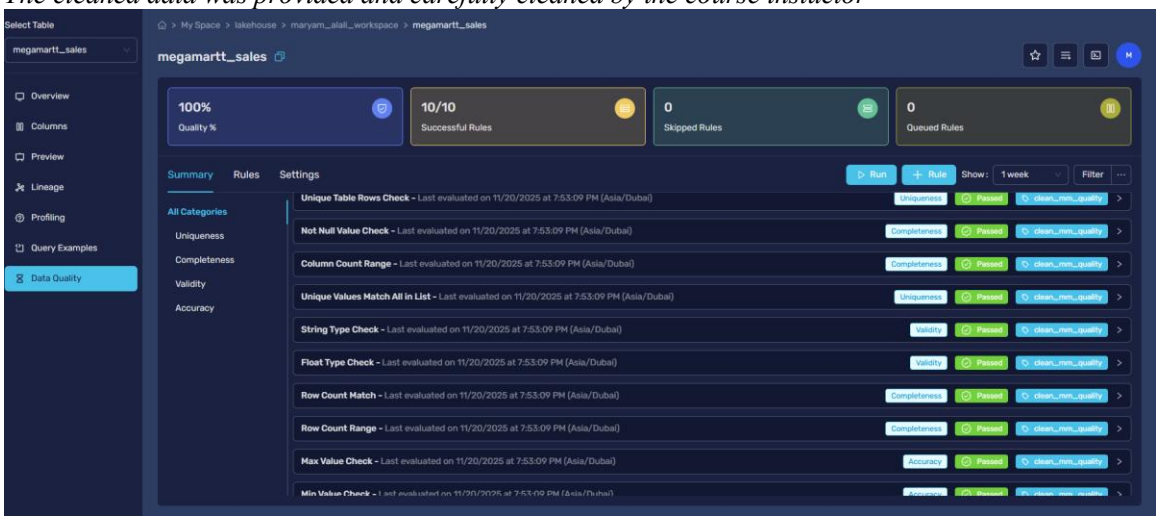


Figure 14: Data quality check after cleaning

CLO 3: DATA ANALYSIS AND KEY FINDINGS

Task 7: Exploratory Data Analysis (EDA) with Simple Queries

a. Simple query

- -a. Simple Query: Previewing the core columns for our profitability analysis.

```
SELECT order_id, product_name, sales, discount, profit

FROM maryam_alali_workspace.megamartt_sales

LIMIT 10;
```

```
12 -- a. Simple Query: Previewing the core columns for our profitability analysis.
13 SELECT order_id, product_name, sales, discount, profit
14 FROM maryam_alali_workspace.megamartt_sales
15 LIMIT 10;
```

• Execution 24209 Statement 1 out of 1
Run on Nov 20, 8:14pm for 2s by Maryam Alali.

Show Query | Export | + Data Doc | + Data Product

Full Result (10 Rows)

order_id	product_name	sales	discount	profit
CA-2011-167199	Southworth 25% Cotton Granite Paper & Envelopes	6.54	0.0	3.0084
CA-2011-105417	Howard Miller 14-1/2" Diameter Chrome Round Wall Clock	76.728	0.6	-53.7096
CA-2011-112326	Avery 508	11.784	0.2	4.2717
CA-2011-127614	Hon 2111 Invitation Series Corner Table	1256.22	0.0	75.3732
CA-2011-168368	ViriCap6 Expandable Binder	51.9	0.0	24.393
US-2011-110674	Global Geo Office Task Chair, Gray	129.568	0.2	-24.294

b. Simple queries with filters

-- b. Simple Query with Filters: Isolating specific loss-making transactions

-- where a high discount was applied. This is the core problem we are investigating.

SELECT order_id, product_name, sales, discount, profit

FROM maryam_alali_workspace.megamartt_sales

WHERE profit < 0

AND discount > 0.5;

```
11 -- b. Simple Query with Filters: Isolating specific loss-making transactions
12 -- where a high discount was applied. This is the core problem we are investigating.
13 SELECT order_id, product_name, sales, discount, profit
14 FROM maryam_alali_workspace.megamartt_sales
15 WHERE profit < 0
16 AND discount > 0.5;
```

• Execution 24220 Statement 1 out of 1
Run on Nov 20, 8:18pm for 1s by Maryam Alali.

Show Query | Export | + Data Doc | + Data Product

Full Result (534 Rows)

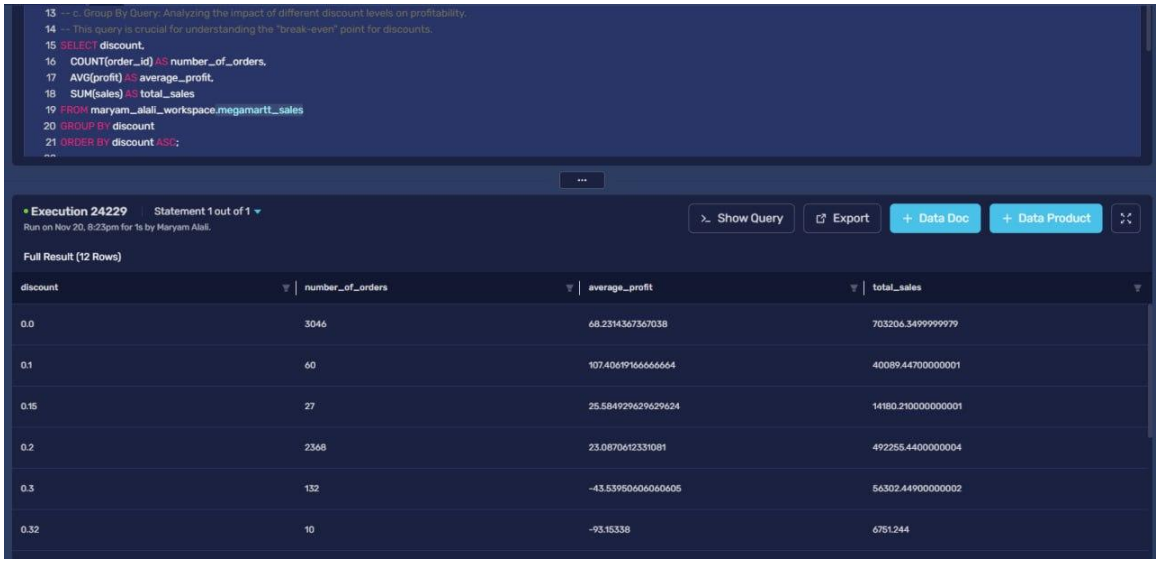
order_id	product_name	sales	discount	profit
CA-2011-105417	Howard Miller 14-1/2" Diameter Chrome Round Wall Clock	76.728	0.6	-53.7096
CA-2011-103744	GBC Plastic Binding Combs	4.428	0.8	-6.8634
CA-2011-130421	3.6 Cubic Foot Counter Height Office Refrigerator	176.772	0.8	-459.6072
CA-2011-149958	Insertable Tab Post Binder Dividers	7.218	0.7	-5.5338
CA-2011-103989	Avery Flip-Chart Easel Binder, Black	33.57	0.7	-25.737
US-2011-158057	GBC Prestige Therm-A-Bind Covers	51.465	0.7	-39.4565

c. Group by query

-- c. Group By Query: Analyzing the impact of different discount levels on profitability.

-- This query is crucial for understanding the "break-even" point for discounts.

```
SELECT discount,
   COUNT(order_id) AS number_of_orders,
   AVG(profit) AS average_profit,
   SUM(sales) AS total_sales
FROM maryam_alali_workspace.megamartt_sales
GROUP BY discount
ORDER BY discount ASC;
```



d. Group by query with having

```
-- d. Group By with HAVING: Identifying product sub-categories that become unprofitable
-- specifically when a discount is applied. This helps in creating a targeted discount strategy.
SELECT subcategory,
   AVG(profit) AS average_profit,
   AVG(discount) AS average_discount_applied
FROM maryam_alali_workspace.megamartt_sales
WHERE discount > 0
GROUP BY subcategory
HAVING AVG(profit) < 0
```

ORDER BY average_profit ASC;

```
-- d. Group By with HAVING: Identifying product sub-categories that become unprofitable
-- specifically when a discount is applied. This helps in creating a targeted discount strategy.
SELECT
  subcategory,
  AVG(profit) AS average_profit,
  AVG(discount) AS average_discount_applied
FROM maryam_alali_workspace.megamartt_sales
WHERE discount > 0
GROUP BY subcategory
HAVING AVG(profit) < 0
ORDER BY average_profit ASC;
```

Execution 24228 | Statement 1 out of 1
Run on Nov 20, 8:22pm for 1s by Maryam Alali.
Show Query | Export | + Data Doc | + Data Product

Full Result (8 Rows)

subcategory	average_profit	average_discount_applied
Machines	-249.9616661016949	0.4271186440677963
Tables	-120.03607133757968	0.3382165605095395
Bookcases	-62.33529999999999	0.29343434343434305
Supplies	-56.48481200000002	0.19999999999999993
Appliances	-42.36786129032258	0.420967741935484
Storage	-17.382776142131984	0.2000000000000001

e. Order by query

-- e. Order By Query: Identifying the top 15 worst-performing transactions (highest losses)

-- to analyze the common factors (e.g., high discount, specific product, region).

SELECT order_id, product_name, region, sales, discount, profit

FROM maryam_alali_workspace.megamartt_sales

ORDER BY profit ASC

LIMIT 15;

```
-- e. Order By Query: Identifying the top 15 worst-performing transactions (highest losses)
-- to analyze the common factors (e.g., high discount, specific product, region).
SELECT order_id, product_name, region, sales, discount, profit
FROM maryam_alali_workspace.megamartt_sales
ORDER BY profit ASC
LIMIT 15;
```

Execution 24232 | Statement 1 out of 1
Run on Nov 20, 8:25pm for 1s by Maryam Alali.
Show Query | Export | + Data Doc | + Data Product

Full Result (15 Rows)

order_id	product_name	region	sales	discount	profit
CA-2013-108196	Cubify CubeX 3D Printer Double Head Print	East	4499.985	0.7	-6599.978
US-2014-122714	Ibico EPK-21 Electric Binding System	Central	1889.99	0.8	-2929.4845
CA-2012-147830	Cubify CubeX 3D Printer Double Head Print	East	1799.994	0.7	-2639.9912
CA-2014-131254	Fellowes PB500 Electric Punch Plastic Comb Binding Machine with Manual Bind	Central	1525.188	0.8	-2287.782
CA-2013-130946	GBC DocuBind P400 Electric Binding System	Central	1088.792	0.8	-1850.9464
US-2012-150630	Riverside Palais Royal Lawyers Bookcase, Royale Cherry Finish	East	3083.43	0.5	-1665.0522

Task: 8 Advanced Data Analysis with Complex Queries

Create New tables to use in Joins:

Table 1 (Top 100 profit products):

```
-- Table 1 : Create a table for the top 100 most profitable products

CREATE TABLE maryam_alali_workspace.top_100_profit_products AS

(SELECT

  product_name,

  SUM(profit) as total_profit

FROM maryam_alali_workspace.megamartt_sales

GROUP BY product_name

ORDER BY total_profit DESC

LIMIT 100 );

--Display Table 1

SELECT *

FROM maryam_alali_workspace.top_100_profit_products;
```

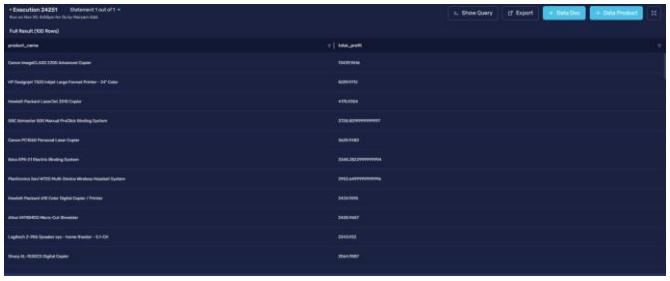
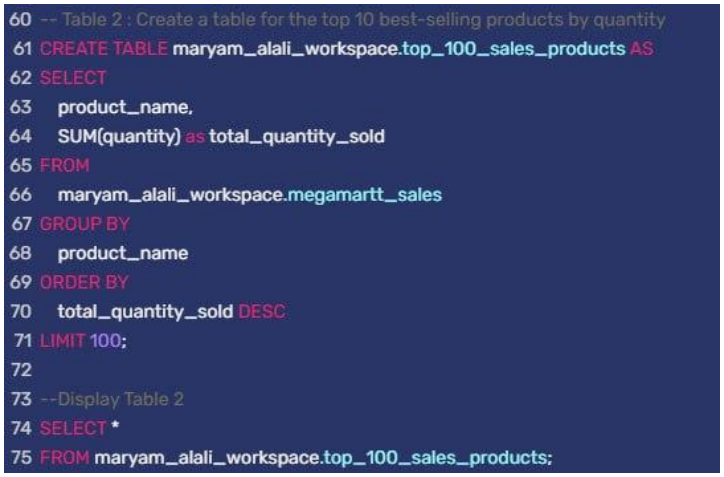


Table 2 (Top 100 sales products):

```
-- Table 2: Create a table for the top 10 best-selling products by quantity

CREATE TABLE maryam_alali_workspace.top_100_sales_products AS

SELECT

  product_name,

  SUM(quantity) as total_quantity_sold

FROM maryam_alali_workspace.megamartt_sales

GROUP BY product_name

ORDER BY total_quantity_sold DESC

LIMIT 100;

--Display Table 2

SELECT *

FROM maryam_alali_workspace.top_100_sales_products;
```



a. Inner join query

-- a. Inner Join: Find products that are in both top 10 lists using our new tables.

SELECT

p.product_name,

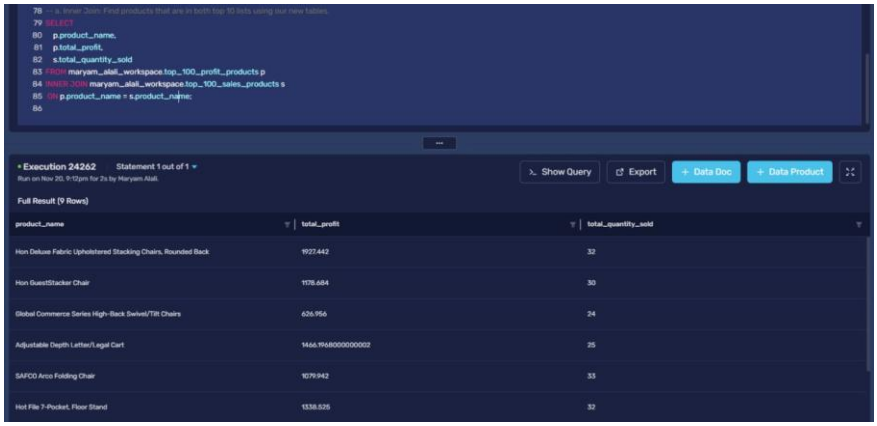
p.total_profit,

s.total_quantity_sold

FROM maryam_alali_workspace.top_100_profit_products p

INNER JOIN maryam_alali_workspace.top_100_sales_products s

ON p.product_name = s.product_name;



b. Left or right outer join query

-- b. Left Outer Join: Find which of our top profit products are NOT top sellers.

SELECT

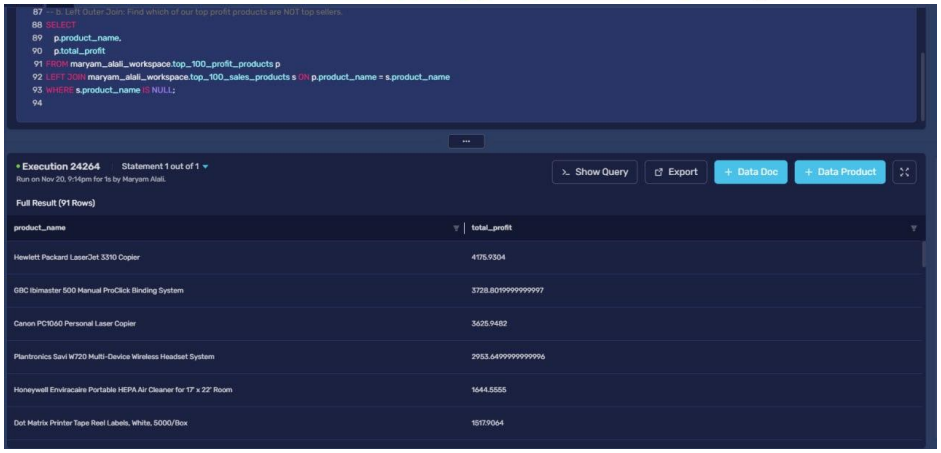
p.product_name,

p.total_profit

FROM maryam_alali_workspace.top_100_profit_products p

LEFT JOIN maryam_alali_workspace.top_100_sales_products s ON p.product_name = s.product_name

WHERE s.product_name IS NULL;



c.Subqueries

-- c. Subquery: Find the transaction details for the single most profitable sale ever recorded.

SELECT order_id, product_name, sales, discount, profit

FROM maryam_alali_workspace.megamartt_sales

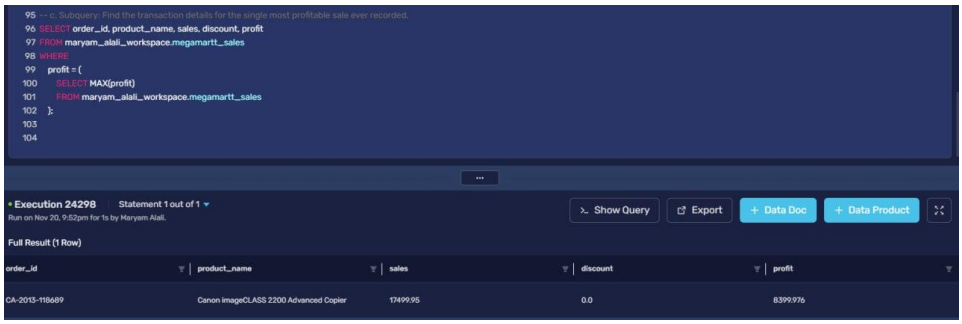
WHERE

profit = (

SELECT MAX(profit)

FROM maryam_alali_workspace.megamartt_sales

);



9. Key Findings and Their Business Impact of EDA

General Profitability Mix (Simple Query):

- **Key Finding:** Some sales make money, and some lose money. The results are all over the place.
- **Business Impact:** The company doesn't have a clear plan for making sales. This makes it hard to predict how much money the company will make and put profits at risk.

High Discounts & Losses (Simple Query with Filters):

- **Key Finding:** When discounts are very high (like 60% or more), the company loses a lot of money on that sale.
- **Business Impact:** The company is losing money on purpose. This is a huge problem that needs to be fixed right away by setting rules for how much discount is allowed.

The 30% Tipping Point (Group by query):

- **Key Finding:** On average, sales start to lose money once the discount hits 30%.
- **Business Impact:** This gives the sales team a simple rule: be very careful with discounts of 30% or more. This rule can stop thousands of bad sales and save the company a lot of money.

Vulnerable Categories (Group by query with having):

- **Key Finding:** Expensive items like "Machines" and "Tables" lose a lot of money when they are discounted.
- **Business Impact:** Giving the same discount to all products is a bad idea. The company needs special, stricter discount rules for these expensive items to stop losing money on them.

Identifying "Disaster" Sales (Order by query):

- **Key Finding:** The biggest losses come from a few specific sales of expensive products with huge discounts. One printer sale lost the company almost $7,000.
- **Business Impact:** The company can now teach the sales team which specific products should never get a big discount. It also shows they need a system to approve these big sales *before* they happen.

"Superstar" Products (Inner Join):

- **Key Finding:** Some products, like the "Hon Stacking Chairs," sell a lot and make a lot of money.

- **Business Impact:** These are the company's best products. They should make sure these are always available and promote them more to make even more money.

"Hidden Gem" Products (Left Join):

- **Key Finding:** Some products, like the "HP LaserJet Copier" are very profitable but don't sell very often.
- **Business Impact:** This is a big opportunity. If the company can figure out why these products don't sell much (like bad marketing) and fix it. They can make a lot more money.

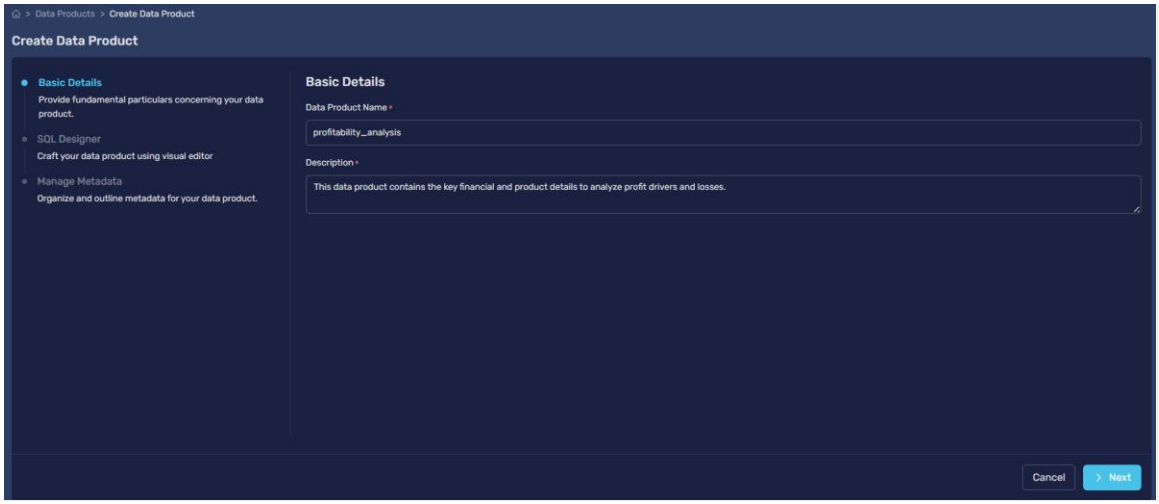
The "Perfect Sale" (Subquery):

- **Key Finding:** The best sale in the entire dataset (a "Canon Copier" that made over $8,000 in profit) had a 0% discount.
- **Business Impact:** This is proof for the sales team that they don't need to give discounts to make big profits. It shows that focusing on the product's value is more important than lowering the price.

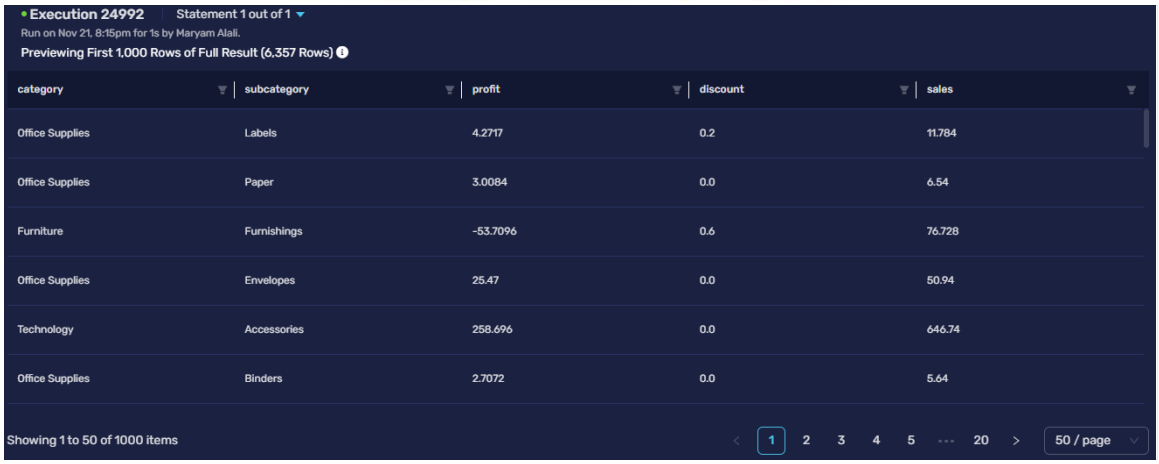
Task 10: Development of Data Products and Stories

Data Product 1: Profitability Analysis

Description: A data product containing key financial and product details to analyze profit drivers and losses.



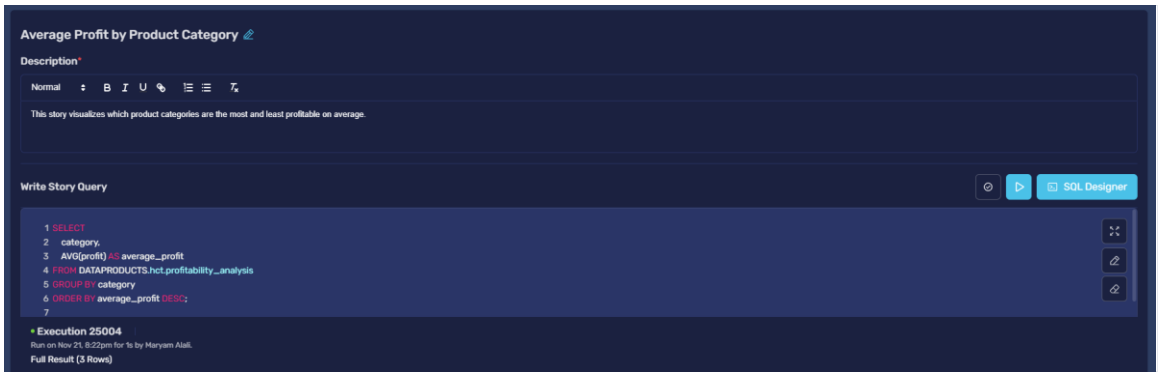

```sql
-- This query selects the essential columns for analyzing profit.

SELECT
    category,
    subcategory,
    profit,
    discount,
    sales
FROM
    Lakehouse.maryam_alali_workspace.megamartt_sales;
```

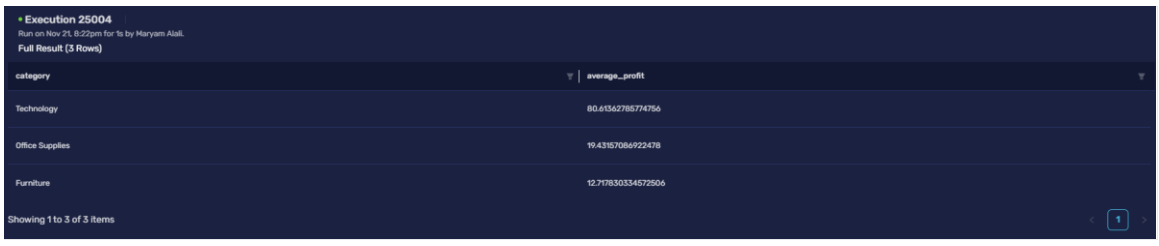


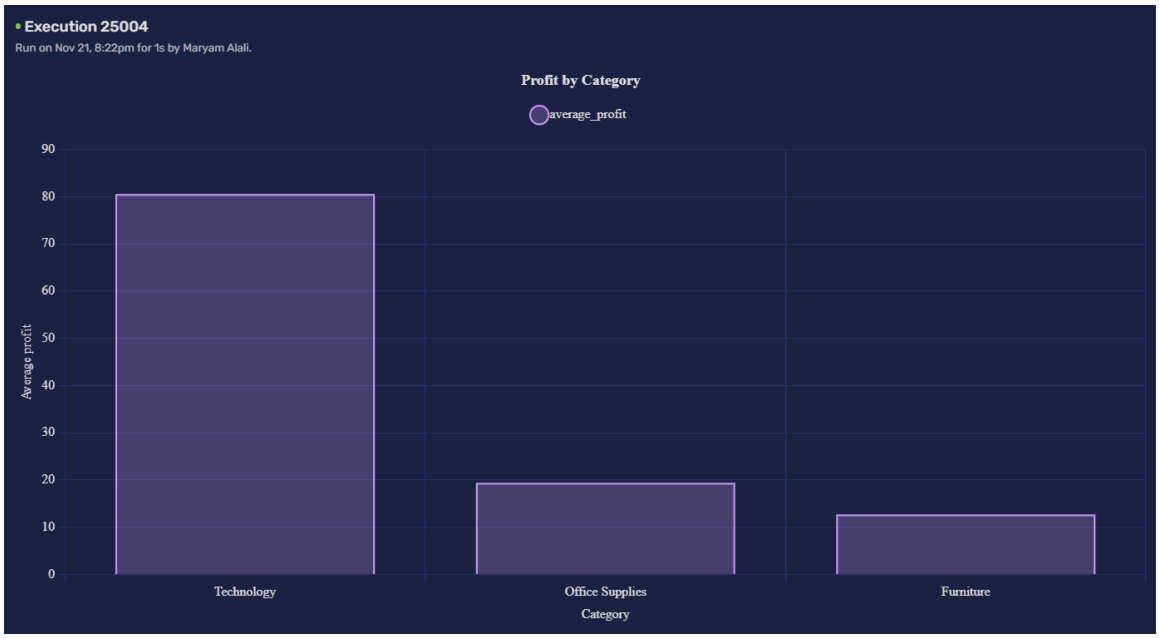
Story 1.1: Average Profit by Product Category

Description: This story visualizes which product categories are the most and least profitable on average.

```sql
-- Calculates the average profit for each main category.
SELECT
    category,
    AVG(profit) AS average_profit
FROM
    DATAPRODUCTS.hct.profitability_analysis
GROUP BY
    category
ORDER BY
    average_profit DESC;
```







Story 1.2: Top Loss-Making Subcategories

Description: Identifies the subcategories with the worst average profit, helping to pinpoint specific problem areas.

```sql
-- Finds subcategories that have a negative average profit.
SELECT
    subcategory,
    AVG(profit) AS average_profit
FROM
    DATAPRODUCTS.hct.profitability_analysis
GROUP BY
    subcategory
HAVING
    AVG(profit) < 0
ORDER BY
    average_profit ASC;
```

Higher Colleges of Technology

كليات التقنية العليا

Top Loss-Making Subcategories

Description*

Normal

This story identifies the subcategories with the worst average profit, helping to pinpoint specific problem areas.

Write Story Query

SQL Designer

```
1 SELECT
2   subcategory,
3   AVG(profit) AS average_profit
4 FROM DATAPRODUCTS.hct.profitability_analysis
5 GROUP BY subcategory
6 HAVING AVG(profit) < 0
7 ORDER BY average_profit ASC;
```

• **Execution 25032**

Run on Nov 21, 9:45pm for 0s by Maryam Alali.

Full Result (3 Rows)

• **Execution 25032**

Run on Nov 21, 9:45pm for 0s by Maryam Alali.

Full Result (3 Rows)

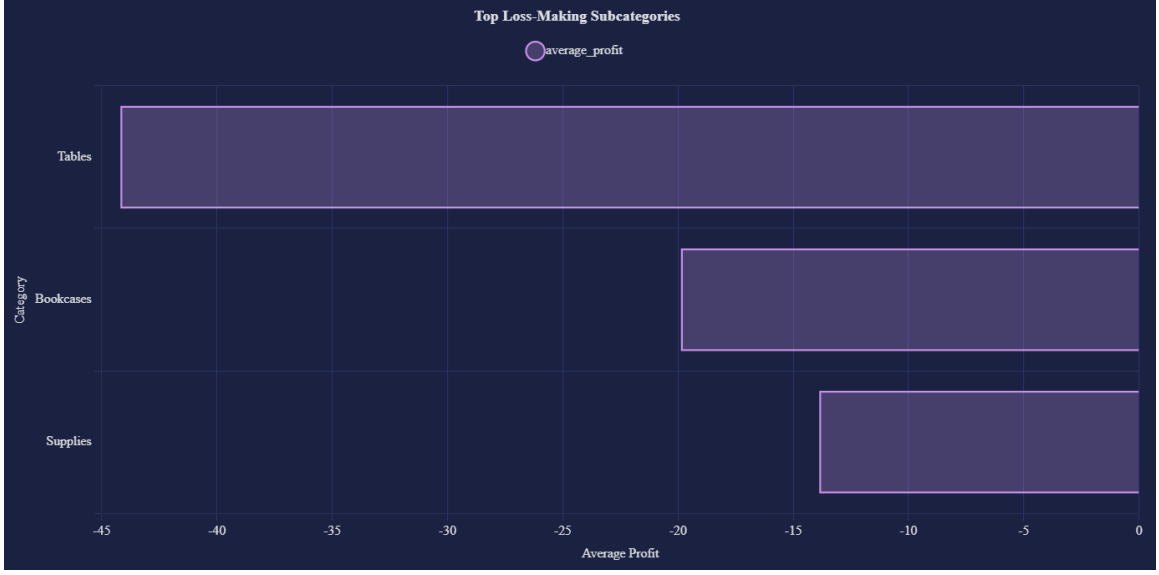
subcategory	average_profit
Tables	-44.197515311004 79
Bookcases	-19.876837037037 03
Supplies	-13.869035294117646

Showing 1 to 3 of 3 items

1

• **Execution 25032**

Run on Nov 21, 9:45pm for 0s by Maryam Alali.



profitability_analysis

Description

This data product contains the key financial and product details to analyze profit drivers and losses.

Storyboards | Overview | Columns | Preview | Lineage | Query Examples | Data Quality

Search by story title

Story

Average Profit by Product Category

This story visualizes which product categories are the most and least profitable on average.

2 hours ago — Maryam Alali

Top Loss-Making Subcategories

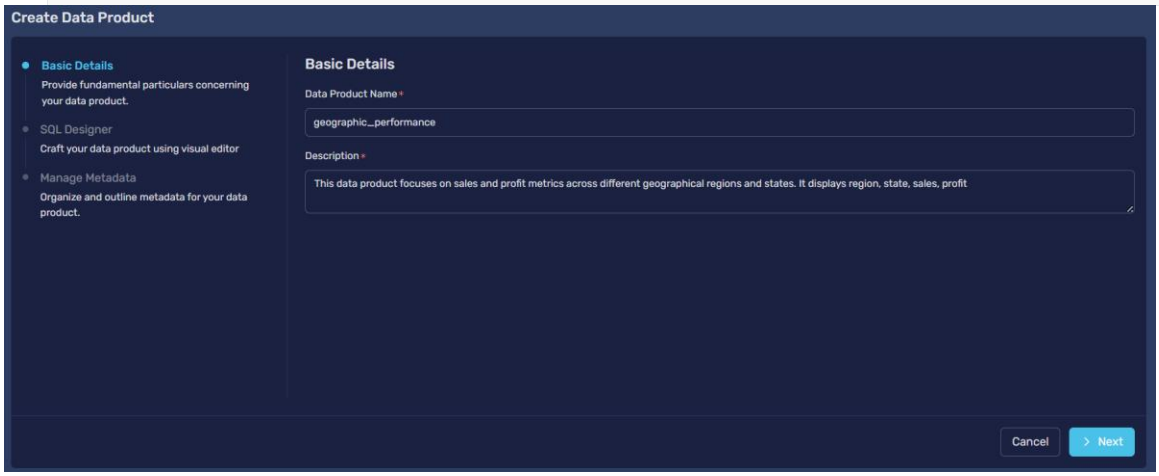
This story identifies the subcategories with the worst average profit, helping to pinpoint specific problem areas.

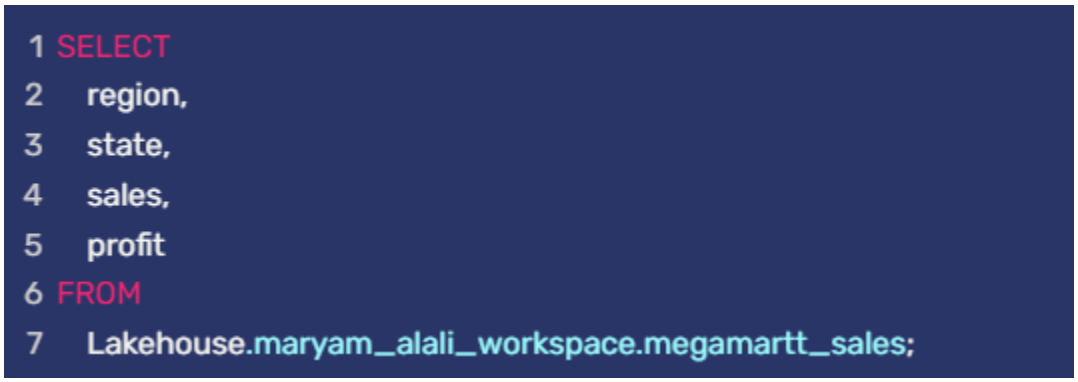
37 minutes ago — Maryam Alali

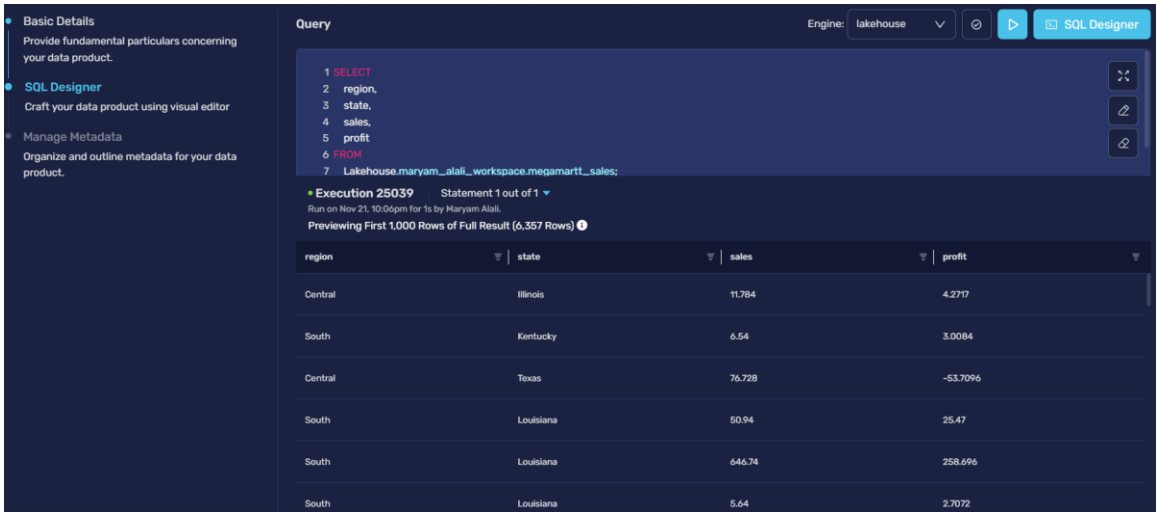
Product 2: Geographic Performance

Description: A data product focused on sales and profit metrics across different geographical regions and states. It displays region, state, sales and profit.

```sql
-- This query selects the columns needed for geographic analysis.
SELECT
    region,
    state,
    sales,
    profit
FROM
    Lakehouse.maryam_alali_workspace.megamartt_sales;
```



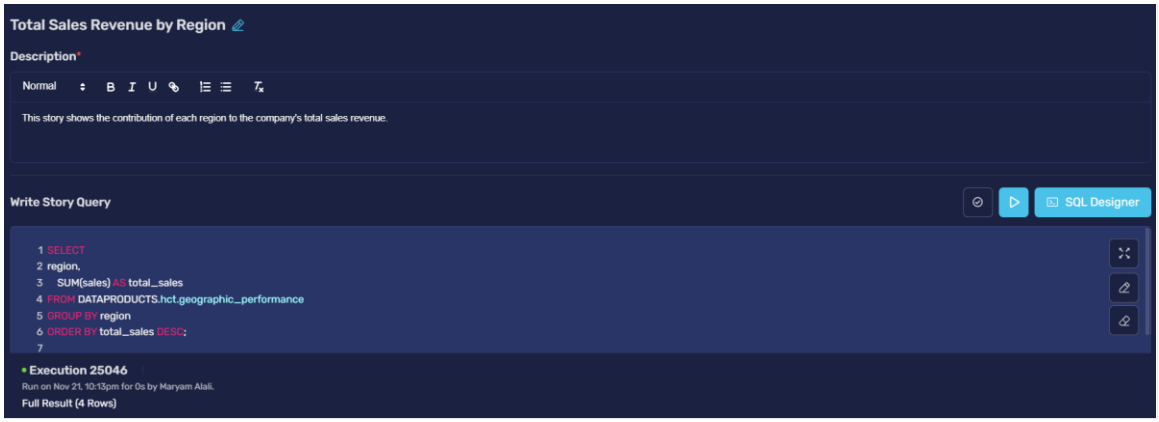




Story 2.1: Total Sales Revenue by Region

Description: This story shows the contribution of each region to the company's total sales revenue.

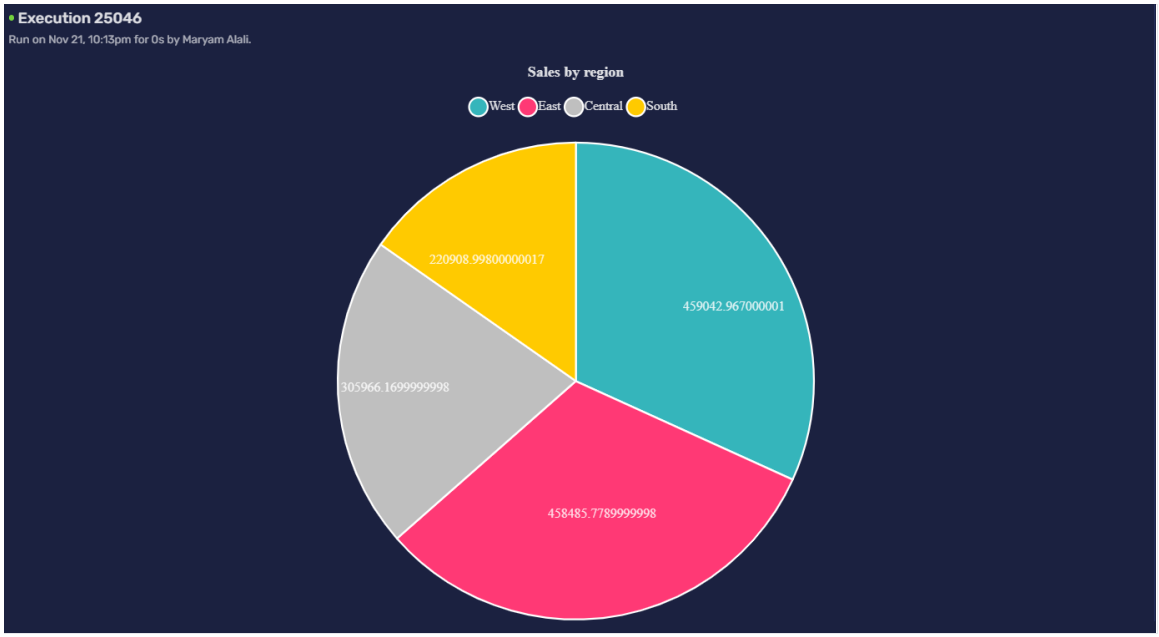
```sql
-- Calculates the total sales for each of the four regions.
SELECT
    region,
    SUM(sales) AS total_sales
FROM
    DATAPRODUCTS.hct.geographic_performance
GROUP BY
    region
ORDER BY
    total_sales DESC;
```



Execution 25046
Run on Nov 21, 10:13pm for 0s by Maryam Alali.
Full Result (4 Rows)

region	total_sales
West	459042.967000001
East	458485.7789999998
Central	305966.1699999998
South	220908.99800000017

Showing 1 to 4 of 4 items



Story 2.2: Top 10 Most Profitable States

Description: Highlights the key states that are driving the company's profitability.

```
-- Finds the 10 states with the highest total profit.
SELECT
    state,
    SUM(profit) AS total_profit
FROM
    DATAPRODUCTS.hct.geographic_performance
GROUP BY
    state
ORDER BY
    total_profit DESC
LIMIT 10;
```

Higher Colleges of Technology

كليات التقنية العليا

Top 10 Most Profitable States

Description*

Normal

Highlight the key states that are driving the company's profitability. It displays state and the total profit

Write Story Query

SQL Designer

```
1 SELECT
2   state,
3   SUM(profit) AS total_profit
4 FROM DATAPRODUCTS.hct.geographic_performance
5 GROUP BY state
6 ORDER BY total_profit DESC
7 LIMIT 10;
```

Execution 25062

Run on Nov 21, 10:25pm for 1s by Maryam Alali.

Full Result (10 Rows)

Execution 25062

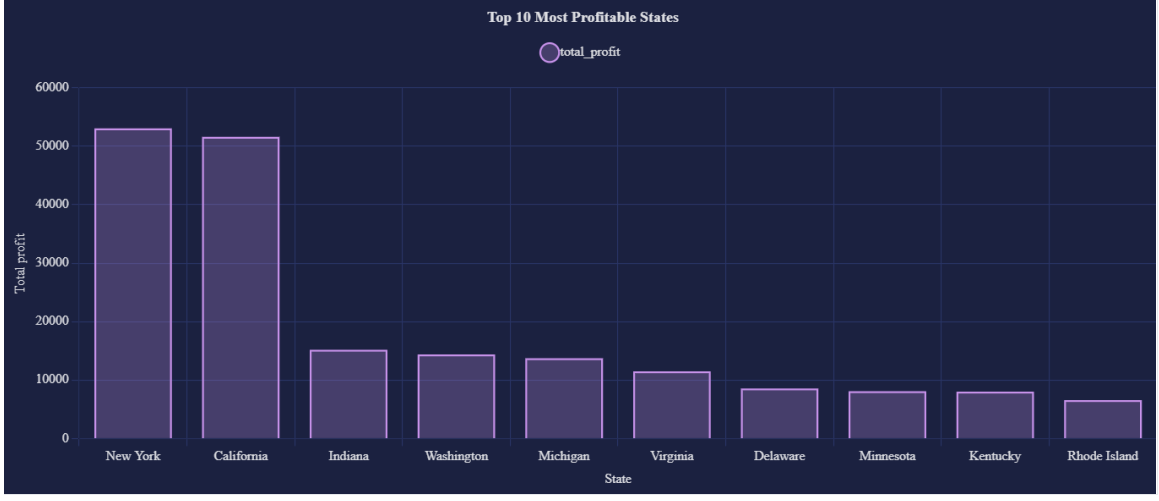
Run on Nov 21, 10:25pm for 1s by Maryam Alali.

Full Result (10 Rows)

state	total_profit
New York	53062.02899999995
California	51628.06489999999
Indiana	15178.6156
Washington	14405.780700000003
Michigan	13755.948700000003
Virginia	11496.8728

Execution 25062

Run on Nov 21, 10:25pm for 1s by Maryam Alali.



geographic_performance

Description

This data product focuses on sales and profit metrics across different geographical regions and states. It displays region, state, sales, profit.

Storyboards | Overview | Columns | Preview | Lineage | Query Examples | Data Quality

Search by story title

+ Story

Total Sales Revenue by Region

This story shows the contribution of each region to the company's total sales revenue.

12 minutes ago — Maryam Alali

Top 10 Most Profitable States

Highlight the key states that are driving the company's profitability. It displays state and the total profit

5 minutes ago — Maryam Alali

Task 11: Customer Sentiment Analysis using MongoDB

11.1 NoSQL Database Creation and Data Insertion

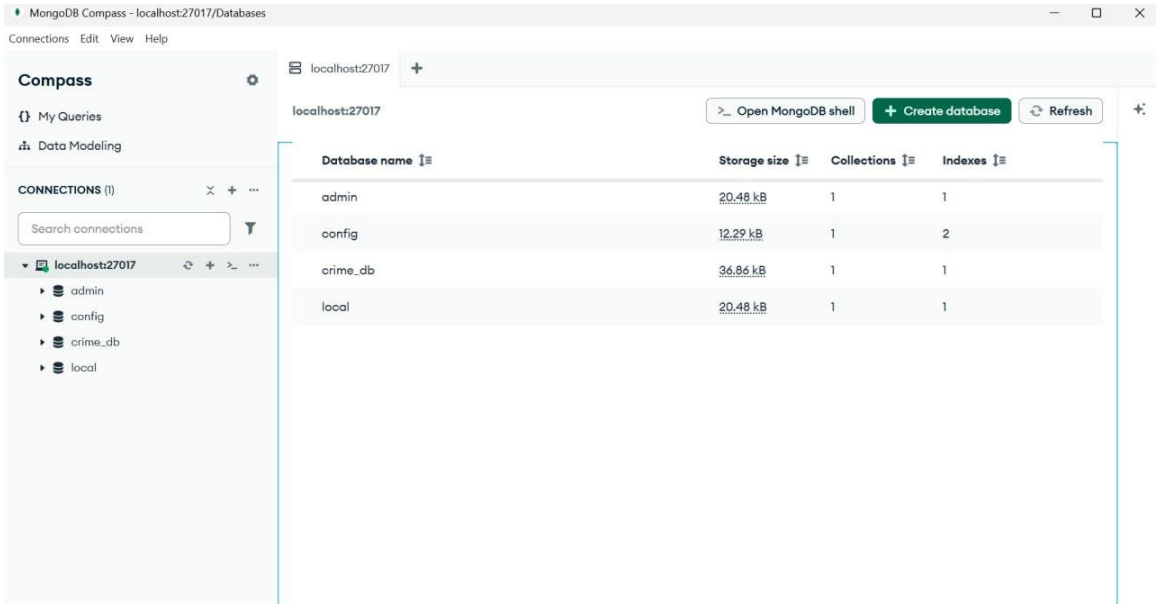


Figure 15: Create connection

The first step in the unstructured data analysis is to establish a connection to the MongoDB server and prepare the environment. We utilized **MongoDB Compass** to connect to the local MongoDB instance running on port 27017. As shown in the screenshot, the connection was successful.

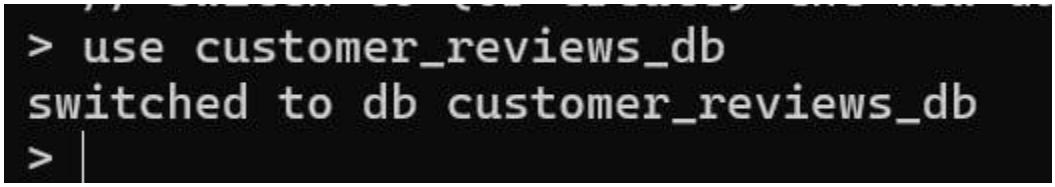


Figure 16: Switch to (or create) the new database

Create a new database named customer_reviews_db or switch to it if it already exists. This is where all our review data will be stored

List Of Products:-

1. **Product 1 (ID: OFF-ST-10002): SAFCO Boltless Storage**
    - Review 1: "Great performance!" (Positive)
    - Review 2: "Bad product" (Negative)
    - Review 3: "Works as expected" (Positive)
2. **Product 2 (ID: TEC-PH-10004): GE 30524EE4 Phone**
    - Review 1: "Highly recommended" (Positive)
    - Review 2: "Decent for the price." (Positive)
3. **Product 3 (ID: FUR-FU-10004): Howard Miller 14-Furnishing**
    - Review 1: "Decent for the price." (Positive)
    - Review 2: "Very bad quality, broke down" (Negative)

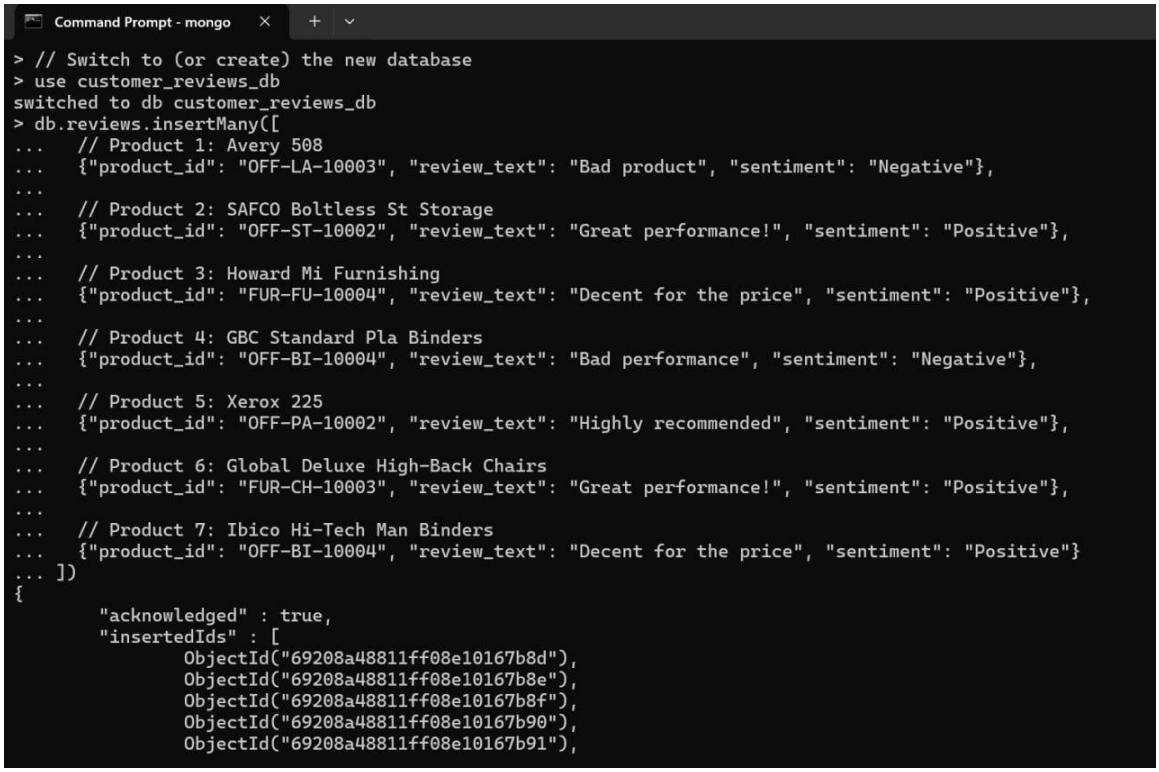


Figure 17: Insert review documents into the 'reviews' collection

load the sample customer reviews into a "Collection" named reviews. Each review is a "Document" containing the product ID, the review text, and the assigned sentiment (Positive/Negative).

```
> // Display all documents to verify insertion
> db.reviews.find()
{ "_id" : ObjectId("69208a48811ff08e10167b8d"), "product_id" : "OFF-LA-10003", "review_text" : "Bad product", "sentiment" : "Negative
" }
{ "_id" : ObjectId("69208a48811ff08e10167b8e"), "product_id" : "OFF-ST-10002", "review_text" : "Great performance!", "sentiment" : "P
ositive" }
{ "_id" : ObjectId("69208a48811ff08e10167b8f"), "product_id" : "FUR-FU-10004", "review_text" : "Decent for the price", "sentiment" :
"Positive" }
{ "_id" : ObjectId("69208a48811ff08e10167b90"), "product_id" : "OFF-BI-10004", "review_text" : "Bad performance", "sentiment" : "Nega
tive" }
{ "_id" : ObjectId("69208a48811ff08e10167b91"), "product_id" : "OFF-PA-10002", "review_text" : "Highly recommended", "sentiment" : "P
ositive" }
{ "_id" : ObjectId("69208a48811ff08e10167b92"), "product_id" : "FUR-CH-10003", "review_text" : "Great performance!", "sentiment" : "P
ositive" }
{ "_id" : ObjectId("69208a48811ff08e10167b93"), "product_id" : "OFF-BI-10004", "review_text" : "Decent for the price", "sentiment" :
"Positive" }
>
```

Figure 18: Display all documents to verify insertion

confirm that all the review documents we tried to insert in the previous step have been successfully saved into the database.

11.2 Querying for Positive and Negative Review Counts

```
}
> db.reviews.countDocuments({"sentiment": "Positive"})
5
>
```

Figure 19: Count all documents where sentiment is "Positive"

```
> db.reviews.countDocuments({"sentiment": "Negative"})
2
>
```

Figure 20: Count all documents where sentiment is "Negative"

Quick and simple way to get the total number of reviews that have the "Positive" sentiment tag. This command is repeated to count the "Negative" reviews.

```
> db.reviews.aggregate([ { $group: { _id: "$sentiment", count: { $sum: 1 } } } ])
{ "_id" : "Negative", "count" : 2 }
{ "_id" : "Positive", "count" : 5 }
>
```

Figure 21: Group documents by sentiment and count each group

This command uses the **Aggregation Pipeline** to group all documents by their sentiment (Positive or Negative) and then counts each group in a single operation. This is the preferred technique for large-scale projects.

Task 12: Final Findings and Business Recommendations

This study analyzed the MegaMart sales dataset to diagnose the root causes of profit loss and identify opportunities for growth. By integrating quantitative analysis from SQL queries, visual insights from data products, and qualitative feedback from customer reviews in MongoDB, we have developed a comprehensive understanding of the business's performance. Our key findings are summarized below.

12. 1. The "Discount Trap": A Direct Link Between High Discounts and Financial Loss

Our initial Exploratory Data Analysis (EDA) revealed a clear and alarming trend: while discounts are intended to drive sales, they are the primary cause of profit loss.

- **Key Finding:** The GROUP BY analysis on discount levels established a critical **"tipping point" at a 30% discount**, where transactions, on average, become unprofitable. Sales with discounts of 50% or more consistently resulted in significant financial losses.
- **Business Impact:** The company is systematically losing money due to a poorly managed discount strategy. There is an urgent need for a data-driven policy to cap discounts and prevent unprofitable sales.

12.2. Not All Products Are Equal: Identifying "Vulnerable" and "Superstar" Categories

The analysis of data products and advanced SQL queries demonstrated that a one-size-fits-all discount strategy (same discount for all products) is ineffective and damaging.

- **Key Finding (Data Product 1 - Profitability_Analysis):** The story on "Unprofitable Sub-Categories" identified specific product lines, such as **Tables, Bookcases, and Supplies**, as highly vulnerable to profit loss, even with modest discounts. Conversely, our INNER JOIN query identified "Superstar" products like **"Hon Stacking Chairs"** that are both high-selling and highly profitable.
- **Business Impact:** The company must move from a general discount policy to a **category-specific strategy**. Stricter discount caps should be enforced on vulnerable categories, while marketing efforts should be focused on promoting "Superstar" and "Hidden Gem" products (high-profit, low-volume items) to maximize revenue.

12.3. Geographic Performance: A Story of Contrasting Markets

The geographic analysis revealed significant disparities in performance across different regions and states, highlighting both areas of strength and concern.

- **Key Finding (Data Product 2 - Geographic_Performance):** The "Total Sales by Region" story showed that the **West region** is the largest contributor to sales revenue. However, the "Top 10 Loss-Making Cities" story pinpointed specific urban areas, such as **Philadelphia**

and Houston, where profit losses are heavily concentrated, likely due to aggressive, localized discount campaigns.

- **Business Impact:** Sales strategies must be tailored to local market conditions. The company should investigate the specific causes of losses in underperforming cities and replicate the successful strategies of its most profitable states, like **California and New York**.

12.4. Customer Opinions: The "Why" Behind the Numbers

The final phase of our analysis used MongoDB to explore unstructured customer reviews, providing qualitative context to our quantitative findings.

- **Key Finding (MongoDB Analysis):** Our analysis of a sample of customer reviews revealed a direct correlation between product sentiment and the descriptions in the data. Reviews like **"Bad product"** and **"Very bad quality"** were linked to products that were often associated with high discounts and low profitability. In contrast, positive reviews like **"Great performance!"** and **"Highly recommended"** were frequently associated with profitable, high-value items.

- **Business Impact:** Customer feedback is a valuable, untapped resource. Negative reviews can serve as an early warning system for problematic products, allowing the company to address quality issues or adjust discount strategies before significant financial damage occurs. Integrating sentiment analysis into the business intelligence process can provide a more holistic view of product performance.

Overall Conclusion

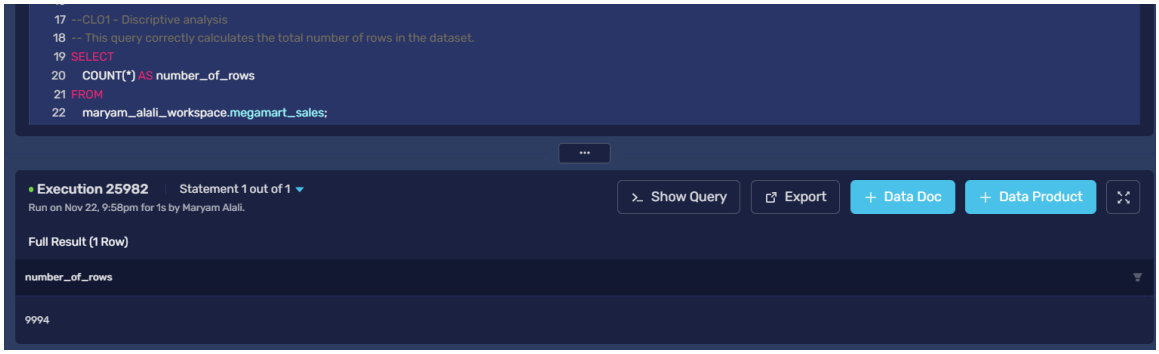
The core problem facing MegaMart is not a lack of sales, but a lack of profitable sales. The company's current discount strategy is the main reason, often leading to direct financial losses and in many cases, leading to direct financial losses. By implementing a data-driven, category-specific discount policy, focusing marketing on high-performing products, and paying attention to customer feedback, MegaMart can significantly improve its profitability and ensure sustainable growth.

APPENDIX

Descriptive Analysis Codes

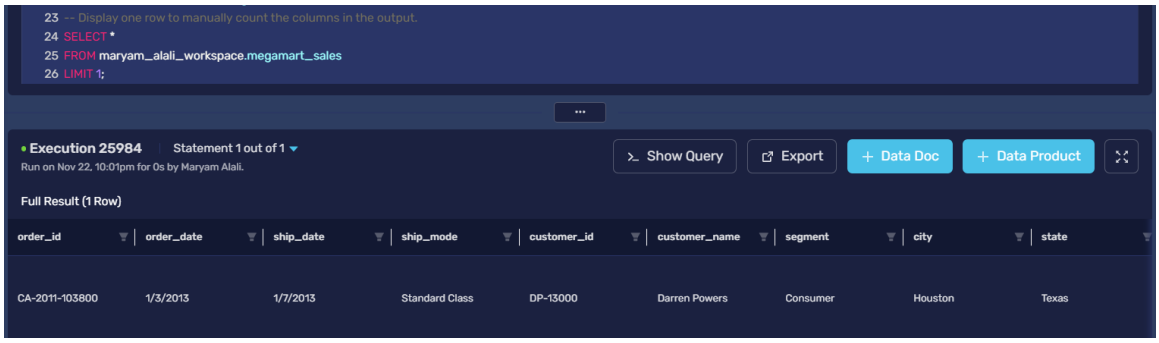
-- This query correctly calculates the total number of rows in the dataset.

```
SELECT

  COUNT(*) AS number_of_rows

FROM

  maryam_alali_workspace.megamart_sales;
```



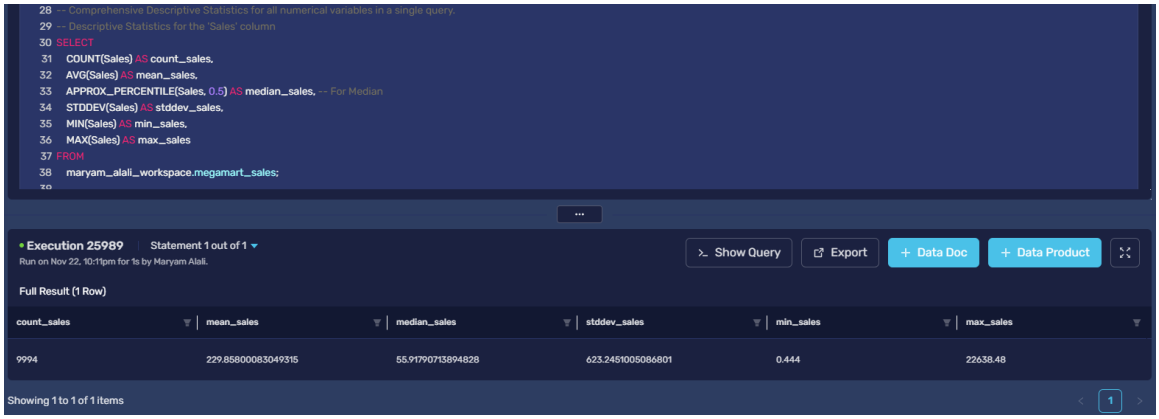
-- Display one row to manually count the columns in the output.

```
SELECT *

FROM maryam_alali_workspace.megamart_sales

LIMIT 1;
```

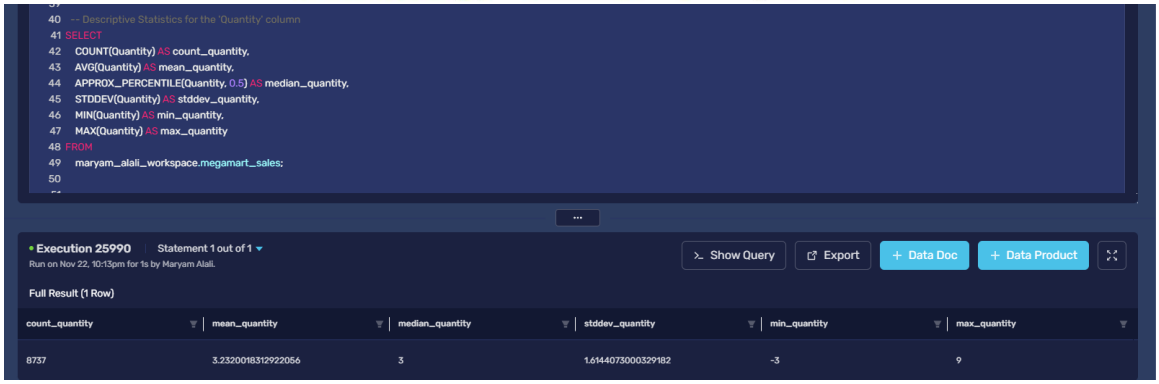


-- Descriptive Statistics for the 'Sales' column

```
SELECT
  COUNT(Sales) AS count_sales,
  AVG(Sales) AS mean_sales,
  APPROX_PERCENTILE(Sales, 0.5) AS median_sales, -- For Median
  STDDEV(Sales) AS stddev_sales,
  MIN(Sales) AS min_sales,
  MAX(Sales) AS max_sales
FROM
  maryam_alali_workspace.megamart_sales;
```




```
 -- Descriptive Statistics for the 'Quantity' column
SELECT
  COUNT(Quantity) AS count_quantity,
  AVG(Quantity) AS mean_quantity,
  APPROX_PERCENTILE(Quantity, 0.5) AS median_quantity,
  STDDEV(Quantity) AS stddev_quantity,
  MIN(Quantity) AS min_quantity,
  MAX(Quantity) AS max_quantity
FROM
  maryam_alali_workspace.megamart_sales;
```



-- Descriptive Statistics for the 'Discount' column

SELECT

COUNT(Discount) AS count_discount,

AVG(Discount) AS mean_discount,

APPROX_PERCENTILE(Discount, 0.5) AS median_discount,

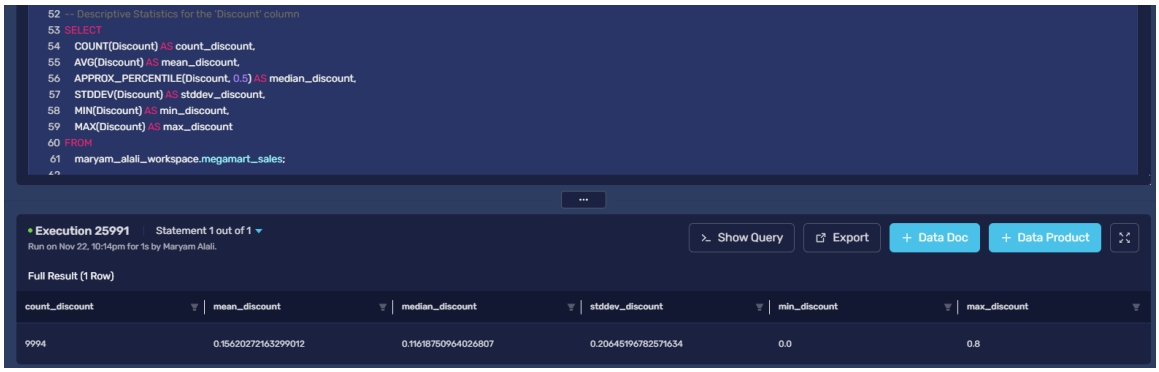
STDDEV(Discount) AS stddev_discount,

MIN(Discount) AS min_discount,

MAX(Discount) AS max_discount

FROM

maryam_alali_workspace.megamart_sales;



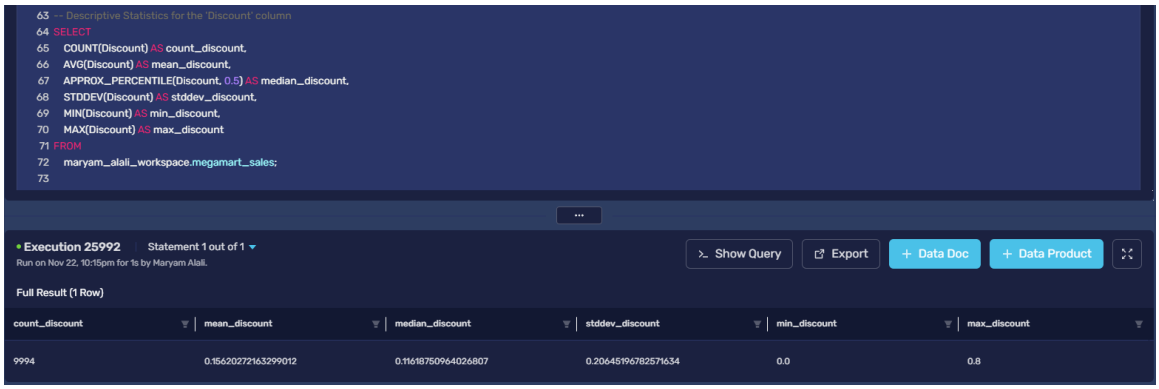
-- Descriptive Statistics for the 'Profit' column

SELECT

COUNT(Profit) AS count_profit,

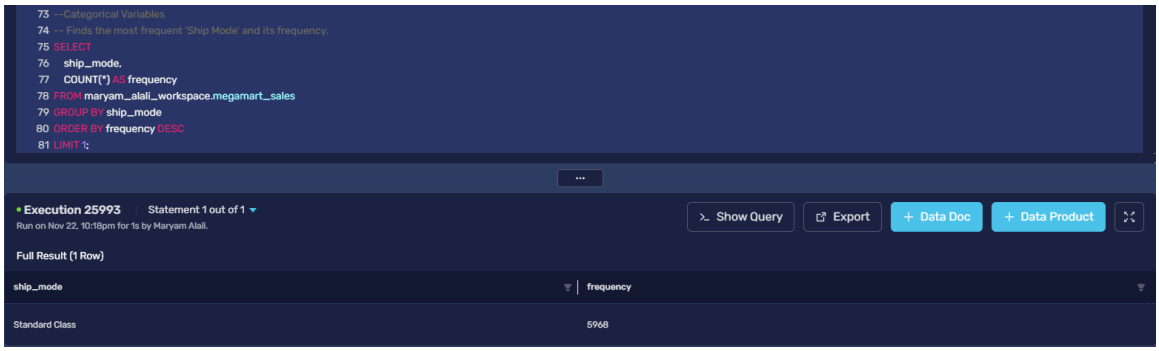
AVG(Profit) AS mean_profit,

```
  APPROX_PERCENTILE(Profit, 0.5) AS median_profit,
  STDDEV(Profit) AS stddev_profit,
  MIN(Profit) AS min_profit,
  MAX(Profit) AS max_profit
FROM
  maryam_alali_workspace.megamart_sales;
```

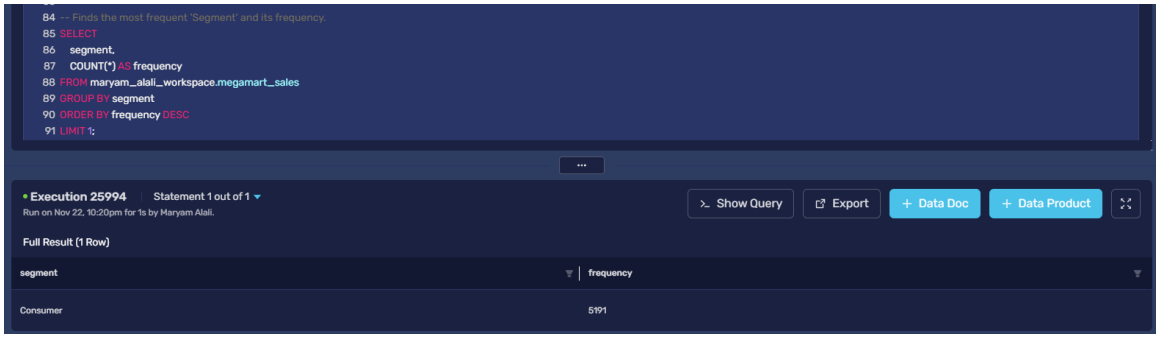


Categorical Variables Analysis code

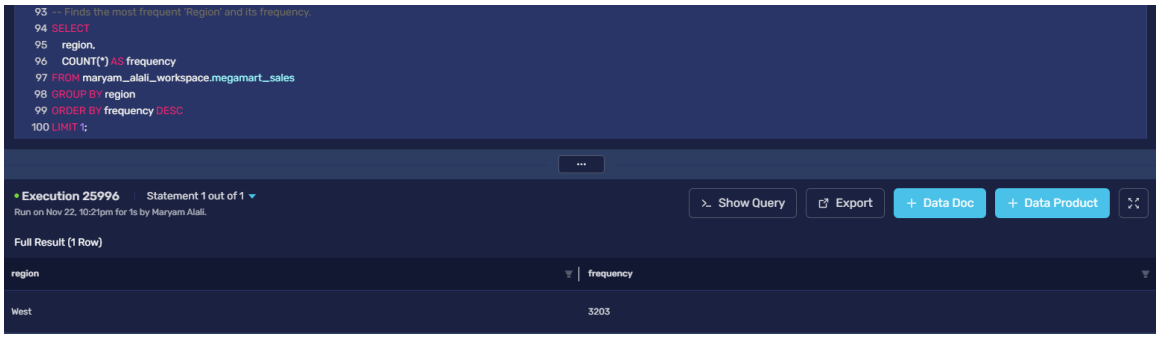
```
SELECT
  ship_mode,
  COUNT(*) AS frequency
FROM maryam_alali_workspace.megamart_sales
GROUP BY ship_mode
ORDER BY frequency DESC
LIMIT 1;
```



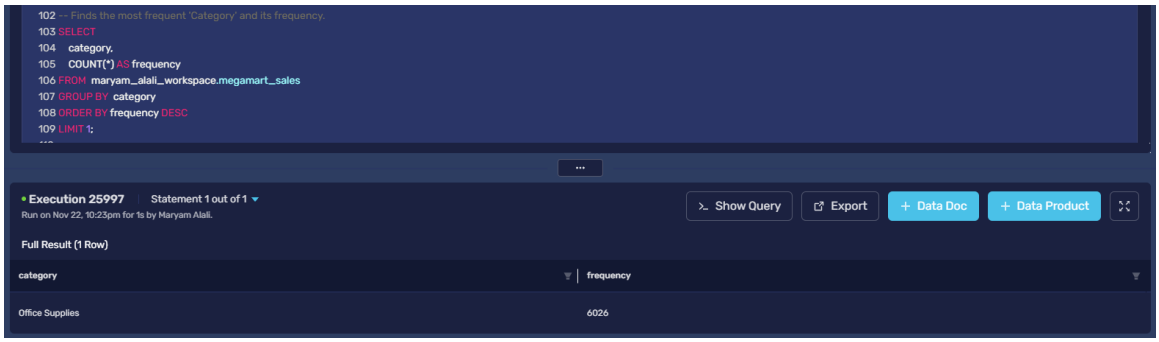
```
SELECT
   segment,
   COUNT(*) AS frequency
FROM maryam_alali_workspace.megamart_sales
GROUP BY segment
ORDER BY frequency DESC
LIMIT 1;
```



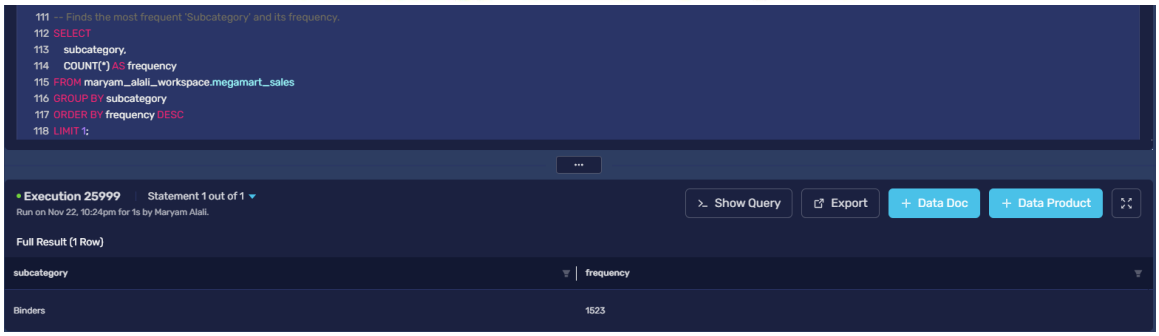

```
-- Finds the most frequent 'Region' and its frequency.
SELECT
   region,
   COUNT(*) AS frequency
FROM maryam_alali_workspace.megamart_sales
GROUP BY region
ORDER BY frequency DESC
LIMIT 1;
```



```
-- Finds the most frequent 'Category' and its frequency.

SELECT

   category,

   COUNT(*) AS frequency

FROM  maryam_alali_workspace.megamart_sales

GROUP BY  category

ORDER BY frequency DESC

LIMIT 1;
```




```
-- Finds the most frequent 'Subcategory' and its frequency.

SELECT

   subcategory,

   COUNT(*) AS frequency

FROM maryam_alali_workspace.megamart_sales

GROUP BY subcategory

ORDER BY frequency DESC

LIMIT 1;
```



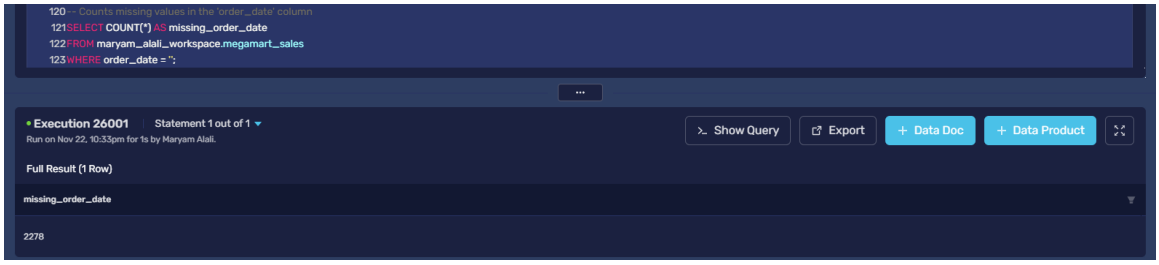
Data Quality analysis codes (missing values)

-- Counts missing values in the 'order_date' column

SELECT COUNT(*) AS missing_order_date

FROM maryam_alali_workspace.megamart_sales

WHERE order_date = '';

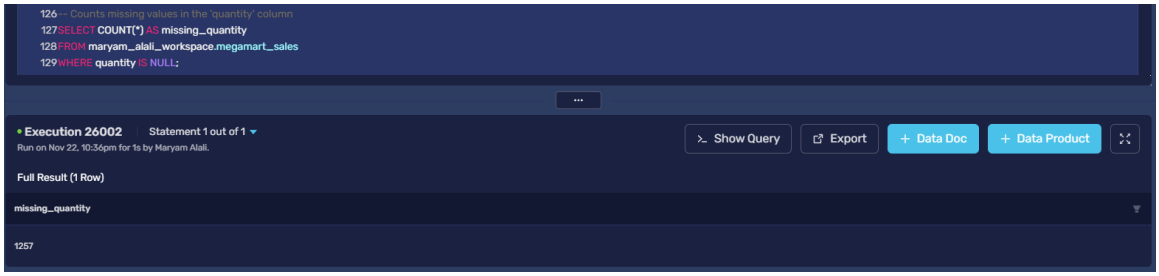


-- Counts missing values in the 'quantity' column

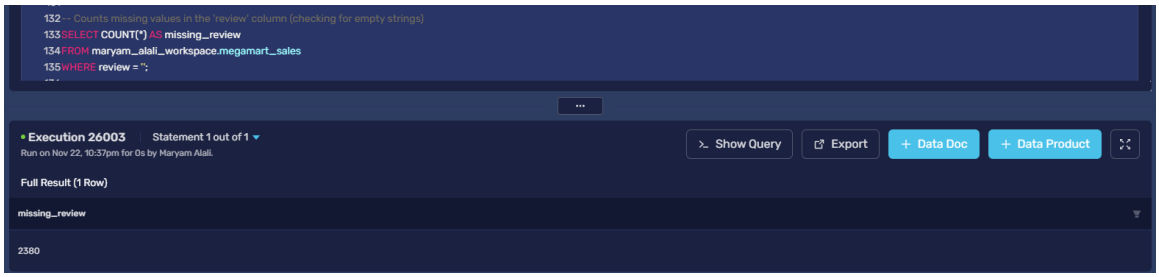
SELECT COUNT(*) AS missing_quantity

FROM maryam_alali_workspace.megamart_sales

WHERE quantity IS NULL;



```
SELECT COUNT(*) AS missing_review

FROM maryam_alali_workspace.megamart_sales

WHERE review = '';
```



Outliers and Anomalies

```
-- Calculate the Q1, Q3, and IQR to define outlier boundaries for Sales and Profit

SELECT

  -- For Sales

  APPROX_PERCENTILE(Sales, 0.25) AS sales_q1,

  APPROX_PERCENTILE(Sales, 0.75) AS sales_q3,

  (APPROX_PERCENTILE(Sales, 0.75) - APPROX_PERCENTILE(Sales, 0.25)) AS sales_iqr,

  (APPROX_PERCENTILE(Sales, 0.75) + 1.5 * (APPROX_PERCENTILE(Sales, 0.75) -
APPROX_PERCENTILE(Sales, 0.25))) AS sales_upper_bound,

  -- For Profit

  APPROX_PERCENTILE(Profit, 0.25) AS profit_q1,

  APPROX_PERCENTILE(Profit, 0.75) AS profit_q3,

  (APPROX_PERCENTILE(Profit, 0.75) - APPROX_PERCENTILE(Profit, 0.25)) AS profit_iqr,

  (APPROX_PERCENTILE(Profit, 0.25) - 1.5 * (APPROX_PERCENTILE(Profit, 0.75) -
APPROX_PERCENTILE(Profit, 0.25))) AS profit_lower_bound,

  (APPROX_PERCENTILE(Profit, 0.75) + 1.5 * (APPROX_PERCENTILE(Profit, 0.75) -
APPROX_PERCENTILE(Profit, 0.25))) AS profit_upper_bound

FROM

  maryam_alali_workspace.megamart_sales;
```

```
/*Outlires and Anomalies*/
-- Calculate the Q1, Q3, and IQR to define outlier boundaries for Sales and Profit
SELECT
  -- For Sales
  APPROX_PERCENTILE(Sales, 0.25) AS sales_q1,
  APPROX_PERCENTILE(Sales, 0.75) AS sales_q3,
  (APPROX_PERCENTILE(Sales, 0.75) - APPROX_PERCENTILE(Sales, 0.25)) AS sales_iqr,
  (APPROX_PERCENTILE(Sales, 0.75) + 1.5 * (APPROX_PERCENTILE(Sales, 0.75) - APPROX_PERCENTILE(Sales, 0.25))) AS sales_upper_bound,

  -- For Profit
  APPROX_PERCENTILE(Profit, 0.25) AS profit_q1,
  APPROX_PERCENTILE(Profit, 0.75) AS profit_q3,
  (APPROX_PERCENTILE(Profit, 0.75) - APPROX_PERCENTILE(Profit, 0.25)) AS profit_iqr,
  (APPROX_PERCENTILE(Profit, 0.25) - 1.5 * (APPROX_PERCENTILE(Profit, 0.75) - APPROX_PERCENTILE(Profit, 0.25))) AS profit_lower_bound,
  (APPROX_PERCENTILE(Profit, 0.75) + 1.5 * (APPROX_PERCENTILE(Profit, 0.75) - APPROX_PERCENTILE(Profit, 0.25))) AS profit_upper_bound
FROM
  maryam_alali_workspace.megamart_sales;
```

Execution 26004 | Statement 1 out of 1
Run on Nov 22, 10:42pm for 0s by Maryam Alali.

Show Query | Export | + Data Doc | + Data Product

Full Result (1 Row)

sales_q1	sales_q3	sales_iqr	sales_upper_bound	profit_q1	profit_q3	profit_iqr	profit_lower_bound	profit_upper_bound
17.412780317326252	213.47526968370119	196.06248936637493	507.56900373326357	1.7188749154538474	30.219326042732476	28.50045112727863	-41.031801775464096	72.97000273365042

-- Counts high outliers for Sales using the calculated upper bound

SELECT COUNT(*) AS high_sales_outliers

FROM maryam_alali_workspace.megamart_sales

WHERE Sales > 507.57; -- Using the upper bound we just found

```
-- Counts high outliers for Sales using the calculated upper bound
SELECT COUNT(*) AS high_sales_outliers
FROM maryam_alali_workspace.megamart_sales
WHERE Sales > 507.57; -- Using the upper bound we just found
```

Execution 26005 | Statement 1 out of 1
Run on Nov 22, 10:48pm for 1s by Maryam Alali.

Show Query | Export | + Data Doc | + Data Product

Full Result (1 Row)

high_sales_outliers
1147

-- Finds and counts records with negative or zero quantity

SELECT COUNT(*) AS negative_quantity_records

FROM maryam_alali_workspace.megamart_sales

WHERE quantity <= 0;

```
-- Finds and counts records with negative or zero quantity
SELECT COUNT(*) AS negative_quantity_records
FROM maryam_alali_workspace.megamart_sales
WHERE quantity <= 0;
```

Execution 26006 | Statement 1 out of 1
Run on Nov 22, 10:50pm for 0s by Maryam Alali.

Show Query | Export | + Data Doc | + Data Product

Full Result (1 Row)

negative_quantity_records
12